CINAMATE

The Operator's Manual

A PRIVATE STUDIO SYSTEM · 10 CHAPTERS

CONTENTS

01	Getting Started & The Studio Gate	3
02	Projects, The Vault & Cloud Sync	7
05	The Timeline I — Script Import & Breakdown	11
06	The Timeline II — Estimation & Exports	17
08	The Producer Suite I — The Budget Top Sheet	23
09	The Producer Suite II — Scheduling, DOOD & Call Sheets	29
12	Set Design & The 3D Builder	34
15	On Set — Production, Dailies & Today	40
17	The Editor I — Cutting	44
19	Tools & Utilities	50

Getting Started & The Studio Gate

Cinamate is a private studio system. It has five keys, one door, and twenty-eight rooms behind it. This chapter is about the door: who holds a key, what the lock protects, how long a key stays warm, and what you are looking at once you are through.

> **About this manual.** Everything here was established by reading the source in this repository, not by operating a running site. The platform is under active development and the code moves faster than this page. Where the manual and the software disagree, the software is what runs. Where a behaviour could not be established from source, this chapter says so rather than guessing.

Two doors, and only one of them opens

A deployment publishes two things. The first is a small public shell: the landing page, `login.html`, the brand assets, and a short allow-list of files the landing itself loads. The second is the studio — and the studio is not published at all. It is bundled *inside* a serverless function: at release time `scripts/deploy_cinamate.mjs` moves everything not on that allow-list into the payload of `netlify/functions/gate.js` and rewrites the redirect rules so those paths resolve to it.

That is the security model in one sentence: **there is no "view source" route into Cinamate.** An anonymous visitor asking for a module page is not shown a page with the buttons greyed out — those bytes are not on the public CDN at all. They exist only inside the function, which reads the cookie before it reads the file.

The sign-in page

`login.html` is a single card headed CINAMATE, with the line *Production access — owners only* beneath it, fields labelled Login and Password, and a button reading **Enter the studio**. Submitting posts the name and password as JSON to `/.netlify/functions/verify-owner`. Two query parameters alter it: `?u=` pre-fills the Login field, and `?to=` sets where you land afterwards, defaulting to `/dashboard.html` and refused unless it resolves to a plain same-origin path — so a crafted link cannot use the sign-in card to bounce you off the site.

On success the page keeps only a display record — owner short and timestamp — in `sessionStorage` under `cinamate.session`. It never sees the session token: that is deliberately absent from the response body, because anything this page's script can read, injected script could read too.

The five owner accounts

Five names are valid and no others. The list appears in `verify-owner.js`, `gate.js` and `projects-sync.js`, and all three check it independently.

LOGIN	PASSWORD LIVES IN
`mz465`	`OWNER_PW_MZ465`
`kz465`	`OWNER_PW_KZ465`
`hz465`	`OWNER_PW_HZ465`
`rz465`	`OWNER_PW_RZ465`
`dz465`	`OWNER_PW_DZ465`

The passwords are Netlify environment variables on the deployed site — not in this repository, not in any page, and not in this manual. A sixth, `OWNER_TOKEN_SECRET`, is the HMAC key sessions are signed and verified with.

When sign-in refuses

Every failure returns the same message — *Invalid name or password* — after the same short random pause, whether the name exists or not, so the response cannot be used to discover which logins are real.

Two throttles sit in front of it. The first counts failures per client address: more than **12 in a 10-minute window** answers `429` with *Too many attempts — wait a few minutes and try again*. That count is held in the function's memory and in a shared Netlify Blobs store, `cinamate-auth`, keyed by a digest of the address rather than the address itself. The second counts failures per *name* across every address: past **20** in the same window, answers for that name are delayed by 1.5 seconds. It is a delay and never a lockout — a lockout would let anyone shut an owner out of their own studio by typing a wrong password often enough. Both fail open: if the shared counter cannot be reached the system falls back to the weaker in-memory limit, because a storage outage must not become a denial of service against the only five people who can get in.

The twelve-hour session

A correct password mints a token of the form `owner:name:expires:hmac`, valid for **12 hours**. It returns as two cookies, both with a 43,200-second lifetime:

COOKIE	CARRIES	READABLE BY PAGE SCRIPTS
`cin_owner`	the signed session token	No — HttpOnly
`cin_who`	the owner short, for UI labels only	Yes

On every request the gate re-derives the HMAC, compares it in constant time, checks the expiry, and checks the name against the five. It reads *every* `cin_owner` the request carries, up to twelve, accepting whichever verifies — so a planted cookie cannot stand in front of your real session and quietly sign you out.

When the twelve hours lapse, nothing dramatic happens; things simply stop being served. A page request is redirected to `/login.html?to=` plus the path you were on, so signing in again returns you where you were. A script, stylesheet or image gets a bare `401 Sign in required`. A call to the studio cloud comes back as *Sign in to use the studio cloud*.

> **Two limits of the expiry, both read from source.** There is no renewal path — a token is minted only for a POST carrying the password — so a long day means signing in twice. And a tab left open can look signed in when it is not: the Operator label comes from a `sessionStorage` record whose age nothing checks. Nothing warns you before a session lapses. If a page starts failing to save, reload it; you will be sent to the sign-in card.

The **Exit** control at the foot of the rail clears the display record, expires both cookies, posts a `logout` so the HttpOnly cookie is cleared authoritatively, deletes the service worker's caches so the next person at that browser starts cold, and returns to the front page. That logout is same-origin only and exempt from the throttle: ending a session on a shared machine must never be rate-limited.

The dashboard, and the rail

Signing in lands you on `/dashboard.html`. The status bar carries a per-session identifier of the form `SES-` plus eight hex characters; **Operator**, showing your owner short; a workspace segment naming your local render bridge if `SB_LocalGPU_v1` holds one and otherwise reading *WORKSPACE READY*; and a UTC clock.

Below it is the Producer Suite — portfolio tiles, a productions table, a stage badge per title. These read this browser's own saved module stores, refresh every thirty seconds and whenever the tab regains focus, and stay blank until real work exists. The header stamp reads *LIVE FROM THIS BROWSER*, and it means it: this is the machine you are sitting at, not the studio's whole catalogue.

> **A stale comment to ignore.** `dashboard.html` still carries a header block describing a simulation engine that ticked invented figures upward on timers. The code below it does the opposite and says so. The tiles are real; the comment is a leftover.

Down the left is the rail, and the rail is how you reach everything: twenty-nine entries in six labelled groups, resolving to twenty-eight distinct module pages — *Sales* is a deep link into the budget module rather than a page of its own. Under 900 pixels the rail becomes a bottom tab bar.

GROUP	ENTRIES (PATH)
Develop	Writer `/writer/`, Boards `/boards/`, Clear `/clearance/`, Invest `/investors/`
Prep	Budget `/producer/`, Deals `/contracts/`, Casting `/casting/`, Scout `/locations/`, Wardrobe `/wardrobe/`, Props `/props/`, Sets `/sets/`, Safety `/safety/`
Shoot	Studio `/timeline/`, Office `/production/`, Money `/finance/`, Tools `/tools/`, Dailies `/dailies/`, Today `/today/`
Post	Editor `/editor/`, Screen `/screening/`, VFX `/vfx/`, Music `/music/`, Post `/post/`
Release	Sales `/producer/#sales`, Deliver `/distribution/`, Fests `/festivals/`, Tax `/taxcredit/`
Always	Workflow `/workflow/`, Projects `/projects/`

One studio, five owners

A *production* is a named slot holding a snapshot of every module store — screenplay, boards, budget, schedule, cast, cut — plus a manifest of its photographs. The vault engine at `projects/lib-vault.js` creates them,

switches between them, and packs them as portable `.cinamate` archive files.

The Projects module also pushes a production to the **studio cloud**, and this is the fact that most changes how the platform feels: the cloud is shared, not personal. One catalogue serves the site and all five owners see all of it. A pushed production is listed to everyone alongside the owner who last saved it; anyone signed in can open it, and anyone signed in can remove it from the shared list — earlier copies are kept, and the confirmation prompt says so. Pushing under an existing name replaces the cloud copy. The transport takes 4 MB of written record per production and 48 MB of media in at most 48 parts, with a 900 KB ceiling per file: stills and short takes, not source masters. Anything over is refused whole, naming the file, rather than uploaded down to whatever happened to fit.

What stays on this device

Everything the platform saves lives in this browser first, under keys of the form `SB_<Module>_v1` in `localStorage` — sixty-odd across the twenty-eight modules — plus binary media in IndexedDB for wardrobe continuity photographs, scout photographs and cutting-room sources. The project index itself is `CIN_Projects_v1`. Two keys are deliberately excluded from every snapshot, archive and cloud push: `SB_LocalGPU_v1`, holding your render bridge address and its API key, and `SB_TMDB_v1`, holding a personal API key. They belong to the machine, not to the film, and nothing carrying a credential travels to another owner.

> **Clearing this browser's site data discards work.** A production not pushed to the studio cloud or exported to a `.cinamate` file exists nowhere else. A production opened on a second machine is a copy, and does not stay in step with the first until somebody pushes again.

Where to begin

Sign in and take the rail from the top. **Writer** is where a production acquires its first real content, and the dashboard tiles stay blank until it does. **Projects**, at the foot of the rail, is where a production is named, snapshotted, archived and shared.

Projects, The Vault & Cloud Sync

A production is not a document you open. It is the whole of this browser's storage, held under one name — and the Projects page is the only place that can move it, copy it, or take it away.

> **On this chapter.** Everything here was established by reading the platform's source rather than by operating it. The software is under active development; if a screen contradicts a paragraph below, the screen is what is running and this page is out of date.

What a production is made of

Two stores hold a film, and they behave differently.

The written record lives in this browser's localStorage, under keys matching `SB_<Module>_v<n>` — the pattern `KEY_RE` in `projects/lib-vault.js`. Script and scenes (`SB_Timeline_v1`), boards, budget (`SB_BudgetSheet_v1`), crew (`SB_Crew_v1`), schedule, clearances, the cut, and some seventy others. A key that does not match that pattern is not part of any production and the vault will not carry it.

The photographs live in IndexedDB, in three databases the vault names explicitly in `BLOB_DBS`: `cinamate_wardrobe` (continuity stills), `cinamate_scout` (location scouting), and `cinamate_cut` (cutting-room sources). Those bytes are never copied into localStorage. A production claims a photograph either because the record carries its project stamp, or because one of its `SB_*` stores still points at that id — which is how a scout photo, which carries no stamp at all, stays attributable to the right film.

The index of every production on the machine is `CIN_Projects_v1`: the active name, and a slot per project holding a full copy of that project's `SB_*` values plus a *manifest* of its media — ids and sizes, no bytes. Bytes stay in IndexedDB deliberately; five photographs written into the one record that indexes every project would exhaust the storage quota and take all of them down together.

Switching between productions

Open takes the outgoing project's live workspace, writes it into that project's slot, and only then rewrites localStorage from the incoming slot. The order matters: the snapshot is persisted before anything is cleared, so an interruption cannot leave a slot pointing at work that has already been thrown away. IndexedDB is not cleared on a switch — the other production's photographs stay on the machine, attributed to it by its manifest.

A name may be reused for nothing: starting a new project under an existing name is refused, and so is renaming onto one. Every module carries the active-project badge from `js/project-badge.js`. If another tab switches productions, open pages lock behind a reload prompt so two films cannot bleed into one another.

Backing up: the .cinamate export

Export backup writes a single file — `format: "cinamate/1"` — containing the project name, a timestamp, every portable `SB_*` value, the media bytes as data URLs, and `blobsMissing`: a list of files the production references that this machine could not produce, recorded at pack time so a later restore can tell "this backup never had them" from "this restore lost them". A cutting-room source above 16 MB is counted and reported, never read into the file.

Four things are deliberately absent.

NOT IN A BACKUP	WHY
`SB_LocalGPU_v1`	This workstation's render-bridge address and its API key. Machine-local, and a credential.
`SB_TMDB_v1`	A personal API key. Same reason.
`CIN_Learn_v1`	Cross-project learning. Outside the `SB_` prefix, so structurally excluded — it belongs to the machine, not to any one film, and it is neither backed up nor synced anywhere.
`CIN_Projects_v1`	The index of every project on this machine. Also outside the prefix, and not one production's to carry.

The two `SB_` exclusions are enforced twice — in the vault's `LOCAL_ONLY` pattern and again in the cloud function — because either side alone is one bug away from publishing a key to every owner.

> **A backup is readable by anyone holding it.** The file is plain JSON, not encrypted and not password-protected. It contains the crew directory as entered: names, rates, telephone numbers, email addresses and emergency contacts, alongside deals, budget actuals and everything else the production has recorded. Treat the file as you would a printed deal memo, and think before sending one over an ordinary channel.

Restoring

> **Restore is destructive.** Writing an archive removes *every* portable `SB_*` key in this browser and then writes what arrived. It is not a merge and there is no undo inside the workspace. The Projects page stashes the current workspace into its own slot before importing — that stash, or a backup file, is the only way back. An archive carrying no production data is refused outright rather than obeyed, because obeying it would clear the workspace and write nothing.

An archive from another owner or machine is treated as untrusted: every string is neutralised before a single key is written, with a short list of prose fields (script, notes, synopsis) kept exactly as typed. That happens in full before storage is touched, so a file that fails the check changes nothing.

The restore reports rather than reassures. It states how many media files were written, how many were unreadable and left out, how many this browser refused to store, and how many the production references but nobody has the bytes for. It claims to be complete only when all four of those are empty.

The studio cloud

The cloud is one shared namespace for five owner accounts, behind the owner login. This is the design, not an oversight: **every owner can list, pull, overwrite and delete every production**, including ones they did not

create. There is no per-owner partition and no private area. Pushing under a name that already exists replaces the cloud copy of that film, and the catalog records who saved it last.

Two protections exist, and they are narrow. A push may declare the version it believes it is replacing; if the cloud has moved on, the save is refused with a conflict rather than burying the newer one. And the badge holds back auto-sync when the cloud copy belongs to another owner and this tab has never taken it over. Neither prevents a deliberate Save to cloud from overwriting a colleague's work.

Versions: a ring of eight

Each push moves the copy it replaces into a rotating ring of eight superseded copies. History lists what is on file, with times and the owner who saved each; restoring puts one back, and the copy that was live at that moment is kept too, so the undo is not itself a way to lose work. A delete also lands in the ring, with a record of who deleted it and when.

> **The ring holds the written record, not the photographs.** Eight copies of every still would be a storage bill rather than a backup policy. Restoring an earlier version brings back the text of the production; it does not bring media back, and the response says so plainly rather than letting an operator infer otherwise.

> **Delete removes the media permanently.** Deleting a production from the cloud sweeps its media parts from the store. The written record stays recoverable through the ring; the photographs do not. This is deliberate and recent: the parts were previously left behind, which meant a production its owner believed deleted could still have its location interiors and wardrobe continuity fetched by anyone who had seen the name. That was a leak that looked like a safety net, so the sweep was made real and the deletion record now states that the media went. Export a `.cinamate` backup before deleting anything you may want the pictures of.

The four-minute auto-sync

Every module page runs a background sync on a four-minute interval (`SYNC_MS`), with a first pass around twenty-five seconds after load and one more attempt when the tab is hidden. It hashes the portable `SB_*` stores, does nothing if they have not changed, checks the cloud catalog, and pushes the active production under this owner's name.

It carries the written record only — never media. The server treats that silence as "this push is not about photographs" and carries the previous media state forward untouched, so the cloud copy's pictures remain whatever the last deliberate Save to cloud uploaded.

Working on two machines, take this literally: whichever machine has a Cinamate tab open is pushing its own idea of the production every four minutes. Pull before you start on the second machine, and close or reload the first — otherwise the version check will start refusing saves, or a quiet machine will push a stale record over a day's work. An archive approaching the cloud's four-megabyte ceiling is skipped by auto-sync entirely and becomes manual territory.

Limits that will stop you

LIMIT	VALUE	APPLIES TO
Written record	4 MB	One production's `SB_*` half in the cloud. Media travels separately and does not count towards it.
One media record	900 KB	Roughly three times a full-size scout JPEG. The cloud carries stills and short takes, not source masters.
One part	1 MB	A single upload request, sized to be cheap to retry.
Parts per production	48	Which is where the 48 MB per-production ceiling comes from — it holds by construction.

Anything over any of these is refused whole, before a single byte goes on the wire, with a message naming the offending file and its size, stating that nothing was sent, and offering the `.cinamate` export instead — which has no size limit. A production is never uploaded down to whatever happened to fit: an archive that arrives without its photographs must not look complete.

> **Not established from source.** The vault engine and the cloud function are exercised by `scripts/test_vault_blobs.mjs` against an in-memory media adapter, so the pinned behaviours above are the engine's. How a real browser behaves under storage pressure — quota refusals, an IndexedDB write aborted mid-restore — is reported by the page when it happens but cannot be characterised by reading, and no figure for it is offered here.

The Timeline I — Script Import & Breakdown

The Timeline is where a screenplay stops being a document and becomes production data: a numbered scene list, a cast, a set list and a page measure. This chapter covers getting the script in and broken down. Estimating and exporting it is the chapter that follows.

> **On this text.** Every statement here was established by reading the source — `timeline/parser.js`, `js/lib-scenes.js`, `timeline/timeline.js` and the two node suites that pin them. The platform is under active development. Where this manual and the software disagree, the software is what runs.

Getting the screenplay in

The page is `timeline/index.html`. Three routes lead into it, and all three end in the same parse.

- **Import** in the top bar, **Import file** on the yellow script bar and **Import Script** in the empty-state panel open the same picker, which accepts `.txt`, `.pdf` and `.fdx`. Anything else is refused with *Unsupported file — use .txt, .fdx, or .pdf.*
- **Paste script**, or the gold **Script** button, opens the screenplay editor: one textarea, which is the screenplay of record. The timeline is derived from it, never the other way round.
- **Re-parse** re-runs the parse over whatever that textarea currently holds.

A `.fdx` is read as XML and only six paragraph types survive: Scene Heading, Action, Character, Dialogue, Parenthetical and Transition. Anything Final Draft calls a Shot, a General element or a note is dropped before the parser sees it. A `.pdf` is read with the vendored pdf.js at `static/vendor/pdf.min.js`: text items are sorted top to bottom then left to right and bucketed into lines within a vertical tolerance of five units. That is a reconstruction, not a transcript, and a PDF with no text layer yields nothing at all.

Because pasted and extracted text arrives with its line breaks destroyed, `normalizeScriptText()` runs on every route. Text counts as flattened when it is one or two lines carrying hundreds of characters, when its average line runs past 130 characters, or when long lines dominate. The unflattener then rebuilds breaks before sluglines, character cues, parentheticals and transitions. **Unflatten** in the editor runs that pass on demand and reports the line count before and after.

> **Import replaces the timeline.** Importing a file parses immediately and overwrites the clips, the parse result and the location bible with no confirmation. **Re-parse** asks first, warning that clips carrying generated video will be rebuilt. **+ New script** asks first, then clears the screenplay, the timeline, the characters and the parse result. Undo restores the previous state within the session only — the history is held in memory and does not survive a reload.

What counts as a scene heading

One grammar answers this for the whole platform: `CScenes.parseSlug()` in `js/lib-scenes.js`. The Timeline delegates to it and keeps no second opinion. A heading opens with an interior/exterior token — `INT`, `EXT`, `EST`, or a

two-sided form (`INT/EXT`, `EXT/INT`, `I/E`, `E/I`, with or without spaces and full stops) — followed by a space, dash, colon or full stop, and then somewhere to be. That separator requirement is what keeps *INTERCUT*, *EXTRAS fill the room* and *Interior designers argue* out of the scene list. A bare `INT.` with no location is not a scene.

LINE	READ AS
INT. KITCHEN - DAY	INT · KITCHEN · DAY
INT KITCHEN - DAY	accepted; the full stop is optional
EST. THE CAPITOL - DAY	EST · THE CAPITOL · DAY
I / E PATROL CAR - DUSK	INT/EXT · PATROL CAR · DUSK
INT. PARIS - LEFT BANK - NIGHT	location keeps both halves
24 INT. FARMHOUSE KITCHEN - NIGHT 24	number 24; right margin stripped
MONTAGE - THE TRAINING	a scene on this page only
1. KITCHEN - DAY	a scene on this page only; number 1

The last two rows matter. `timeline/parser.js` layers three heading forms of its own on top of the shared grammar, because a treatment or a documentary transcript prints no INT./EXT.: *SCENE 12 - KITCHEN*; the all-caps beats FLASHBACK, FLASH CUT, MONTAGE, DREAM, INTERCUT, BACK TO, LATER, TIME CUT and SERIES OF SHOTS; and the numbered form *1. KITCHEN - DAY*. These break scenes here and nowhere else on the platform. A beat heading has no location of its own, so *MONTAGE - THE TRAINING* produces a location row literally named MONTAGE, and a flashback heading one named FLASHBACK. Expect to rename or delete those.

For flattened text the parser keeps one unanchored copy of the interior/exterior token, since the shared expression is anchored to the start of a line and a PDF paste runs three sluglines into one. `scripts/test_parser_slug_agreement.mjs` stops that copy drifting: over 180 assertions require that what the shared model accepts is also found mid-line here, with the same number, location, time of day and interior/exterior value, and that what it rejects is rejected here too. Run it with `node`; it either passes or it names the disagreement.

Scene numbers, and the A-scene

A scene number is not a number. *4A* is a scene number; so is *A4*. Each scene therefore carries six identity fields, and the right one depends on what you are doing with it.

FIELD	MEANING	FOR 4A
`ord`	position in the file; preamble is 0	3
`number`	the script's own printed number, verbatim	4A
`n`	numeric, for arithmetic and filters	4
`label`	what a human reads, what goes on a call sheet	4A
`key`	stable identity, so 4 and 4A cannot collide	4A
`sortKey`	collation only: 4 < 4A < 4B < 5	4.01

The printed number always wins; the ordinal fills in only where the script printed none. Where a shooting script prints its numbers down both margins, the right-hand copy is stripped before anything reads the line, and only when it matches the number on the left, so a location genuinely ending in a numeral survives. A bare number on its own line above a heading — how PDF extraction routinely delivers it — is attached to the heading below, across blank lines. `SC` and `SCENE` prefixes are accepted and stripped.

Two consequences to hold on to. A `FADE IN:` preamble is captured separately and is not a scene, so the first real scene is scene 1, not scene 2. And on a numbered script, asking for scene 3 when the printed numbers run 1, 2, 4A returns nothing rather than the third scene in the file — the platform will not invent a scene the production does not have. `scripts/test_scenes.mjs` pins this with over 150 assertions, including that 4A is a real scene break and that its content is not swallowed into the scene above.

Time of day, and where the parser stops being certain

Time of day is taken from the tail of the heading after the last dash, and only when that tail begins with a recognised word: CONTINUOUS, MOMENTS LATER, LATER, SAME TIME, SAME, MORNING, AFTERNOON, EVENING, MIDNIGHT, PRE-DAWN, DAWN, DUSK, SUNSET, SUNRISE, NIGHT, DAY, MAGIC HOUR. Anything else in that position stays part of the location. CONTINUOUS and LATER are carried through verbatim rather than resolved to a clock time, which is correct — they are relative, and only the script supervisor can settle them.

> **Three cases that come back less certain than they look.** A heading with no time of day leaves the scene's time of day empty — but the clip built from it is stamped `Day`, because the inference falls back to Day when it finds no NIGHT, DAWN or DUSK: the scene says unknown, the clip says Day. Second, *INT. KITCHEN - NEXT DAY* does not register, because NEXT DAY is not a recognised time word: the time of day comes back empty and the location becomes *KITCHEN - NEXT DAY*. Third, *INT. KITCHEN - DAY (FLASHBACK)* parses with a time of day of *DAY (FLASHBACK)* — the flashback rides inside that field rather than being flagged. Confirm all three against the script before anything is scheduled from them.

The platform does model this uncertainty, in `js/lib-storyday.js`, which marks every story-day boundary CERTAIN or UNCERTAIN. CONTINUOUS, SAME TIME, LATER, an explicit *next day* cue and a NIGHT-to-DAY transition are certain; DAY to DAY, NIGHT to NIGHT, DAY to NIGHT, a flashback and any heading with no printed time are guesses, defaulted to the same story day and reported as guesses. Note the scope honestly: of the shipped pages only `wardrobe/index.html` loads that module. The Timeline page does not, and shows no story-day column.

The page measure: eighths

A screenplay page is 55 lines, so one eighth is 55/8 — about 6.875 lines. The measure counts blank lines, because a blank line occupies the page, and counts a long line as the rows it would occupy when wrapped at the standard element widths, 60 columns for action and 35 for dialogue. A scene is measured as its heading plus its body, rounded, with a floor of one: a scene that exists occupies at least an eighth on the board. Page count is the eighths total divided by eight, to one decimal place. The constant is anchored by test, not assumed — 55 lines of content measures as one page, within an eighth.

> **Two measures exist.** The Producer's Estimate panel on this same page does not use the measure above. It carries its own splitter, sizes a scene at roughly five content lines to the eighth with blank lines discarded, and requires the full stop after INT/EXT — so *INT KITCHEN - DAY, EST.* headings and prefixed A-scenes are invisible to it. Its page count will not always match the parser's.

From scenes to clips

The Timeline's visible unit is not the scene but the clip — one previsualised beat. One clip is emitted per shot, and a shot is either a dialogue block or a group of sentences from an action paragraph, gathered until the group reaches six words. Each clip carries its scene index, the scene heading verbatim, the location and time of day lifted from that heading, a shot type and camera move inferred by keyword, and a rotating label from a fixed list (Opening scene, Character intro, Dialogue, Action beat, and so on). The labels are decoration; the heading is load-bearing.

Where the parser finds no heading at all it does not fail: it opens a single fallback scene called `SCENE 1` and files every shot into it. A feature that failed to unflatten therefore appears as one enormous scene with a hundred clips, not as zero scenes.

> **The quality warning is not a safety net.** `parseQualityWarning()` holds two messages — no scene headings found, and many shots in a fallback SCENE 1 — but they require four and six fallback scenes respectively, and the parser creates at most one fallback scene per run. Neither can fire from a normal parse. Judge a parse by the clip count and by the Characters and Locations panels, not by the absence of a warning.

Characters

Names are gathered in several passes over the normalised text. A character cue is an all-caps line of 2 to 40 characters that is not a heading, not a transition, not a parenthetical and does not end in punctuation. Dual dialogue survives in both its forms — the Fountain caret, and the two columns a printed page merges onto one line — so a dual scene does not lose one actor's day. Fountain's `@NAME` and inline *NAME: dialogue* are read as cues. An action-line introduction registers when the parenthetical after the name is descriptive rather than a delivery note: *MERCER (50s, silver hair, ex-military)* yields both the name and the description, while *(V.O.)*, *(CONT'D)* and *(whispering)* yield nothing. Titled names (Mr, Dr, Det, Agent, Sgt, Officer, Captain) are picked up separately, and background roles such as FLIGHT ATTENDANT come in through a looser gate. Against all of that, a stop-list rejects the words that look like names and are not: INT, EXT, FADE, CUT, the times of day, the common location nouns, weather. The result is deliberately conservative and will miss people; **+ Add Character** is how you put them back.

Each character holds a description, body type, wardrobe, voice profile, default emotion, cast role (lead, supporting, background, crowd, voice_only), face lock, lip-sync and a reference image. Role is inferred — a one-word name with dialogue reads as a lead, a multi-word name without dialogue as background. Typing into Description or Wardrobe marks that field as yours and the enricher will not overwrite it. **Sync from parse** re-runs the local extraction; signed in, it also requests an appearance-only character bible from the enrichment agent, falling back to the local extract when that is unavailable. Parsing itself needs no network.

> **Deleting a character is not local to the list.** The control asks for confirmation, then removes that character from the list and from every clip that referenced them.

Locations

A location's key is its heading location, upper-cased with whitespace collapsed, so the INT. and EXT. of one set share an entry. The bible is assembled from three sources in order — the clips, the parsed scenes, and the screenplay text, the last of which also honours explicit `Location:` lines. Each entry carries a name, a description seeded from the heading, the indices of the clips that play there, a lock flag, a reference plate and a consistency phrase injected into the prompt when the location is locked.

> **Sync from clips rebuilds the bible from nothing.** Both a fresh import and the Locations panel's sync button empty the location bible and reconstruct it. Rebuilt entries are unlocked, with no plate and no consistency phrase, so locks, uploaded plates and phrases entered by hand are not carried across. Lock a location after the breakdown is settled, not before.

Editing, splitting, merging and reordering

The screenplay text is the record, so the honest answer to "how do I split a scene" is: add the slugline in the script editor and re-parse. To merge two scenes, delete the second slugline and re-parse. There is no scene-level split, merge or renumber command on this page. Scene identity comes from what the script prints, which is the behaviour you want when the call sheet, the slate and the sides all have to agree.

At clip level the controls are **+ Clip**, which appends an empty beat; **Duplicate clip**, which copies the selected clip as a draft with its video link cleared; drag and drop on the clip row to reorder; and **Undo** / **Redo**, holding up to fifty steps for the session. Reordering renumbers every clip and reassigns its identifier from the new position, but does not renumber scenes: each clip keeps the heading it was built from. There is no delete-clip control — to remove material, edit the screenplay and re-parse.

The right-hand inspector edits one clip: its scene description, its emotion, and three groups of prompt parameters — Scene and setting (location, time, weather, season), Camera (angle, film grade, colour, saturation) and Atmosphere (lighting, mood, FX, sound), each with a toggle deciding whether it reaches the prompt. Editing here changes the clip alone: it does not write back to the screenplay, and the next re-parse overwrites it.

What is stored, and under which key

Everything on this page lives in this browser's local storage. None of it is a server-side record, and clearing site data clears the breakdown.

KEY	HOLDS
`SB_Timeline_v1`	the project: clips, characters, location bible, global settings, assembly, parse result, project name, screenplay text
`SB_Editor_embed_v1`	the embedded editor's own timeline state
`SB_LocalGPU_v1`	local generation bridge URL and key, this browser only
`SB_Timeline_script_hint_v2`	a flag marking the one-time hint as shown
`SB_Scenes_v1`	the shared scene store defined by `CScenes`

One honest note on that table. `SB_Scenes_v1` is the canonical scene register — its shape, its helpers and its key are all defined and tested — but no shipped page was found to write it. The department modules that need scenes call the shared splitter on their own copy of the script text instead.

Save project writes the whole state out as a file, screenplay and parse result included, and **Load project** reads one back. That is the only copy of a breakdown that survives a cleared browser; take one before re-parsing a script that has had a day of hand work put into it.

The Timeline II — Estimation & Exports

A broken-down script is a list of obligations. This chapter is about turning that list into a number a financier will read, and then getting the work back out in a shape another machine will accept. Both halves are arithmetic on what Chapter 05 produced; neither invents anything the script did not already say.

> **How this chapter was written.** Every statement below was established by reading the source files it names — no part of the platform was operated to produce it. Development continues, and if the software in front of you says something different from this page, believe the software.

Two estimates in one panel

The *Producer's Estimate* panel on the Timeline page (`timeline/index.html`, the `budgetPanel` disclosure) is drawn by `timeline/timeline-budget.js`, which exposes itself as `SBBudget`. It answers two questions from the same parsed script: what the AI rough-draft preview will bill and how long that run takes, and what the script would cost to shoot for real as a tiered line-item budget with a shoot-day schedule attached. Tier selections persist under the storage key `SB_Budget_v1`, and the panel recomputes only when the script, clips, characters, locations, model, quality or clip duration changed.

The measurement pass

`analyze()` runs first and everything downstream depends on it. It prefers `scriptText` when that is longer than 200 characters, otherwise it reassembles the text from the clips' headings, descriptions and dialogue.

Pages come from eighths. The text is split at sluglines (`INT.`, `EXT.`, `INT/EXT.`, `I/E.`) and each scene sized at five content lines to the eighth — the constant is `LINES_PER_EIGHTH = 5`. Pages is the sum of eighths over eight. The fallbacks matter: with fewer than two sluglines, page count reverts to a word count at roughly 200 words per page, and with barely any text, to clip seconds over sixty. Runtime in minutes is set equal to page count — the one-page-one-minute rule of thumb, stated as such.

Scenes and locations are read off the headings: each is classified interior or exterior, tested for `NIGHT`/`DUSK`/`EVENING`/`MIDNIGHT`/`PRE-DAWN`, and reduced to a location name. Unique locations is the larger of that set and your location bible. **Cast is ranked, not counted** — characters are ordered by how many clips they appear in, the top one or two become leads, the next six at most supporting, everything beyond that a day player. A script with forty speaking parts still models two leads and six supporting roles.

Budget drivers are keyword families, twelve of them, each a regular expression counted over the whole script and weighted out of ten: fire and explosions and VFX/creatures at 9; stunts, water work and period setting at 8; vehicle action 7; weapons and crowds 6; animals 5; weather and child actors 4; aerial work 3. The *complexity* figure shown as `n/100` caps each driver at ten hits and normalises against 72, the maximum weight sum. Genre is inferred the same way from seven keyword families, with Drama as the floor and Action taking over if the physical-action drivers outscore it.

> **These are word matches, not comprehension.** A character named Hunter, dialogue about a storm that never appears on screen, or a metaphorical "explosion of laughter" all count. Read the driver chips before you trust the schedule they inflated.

The schedule the money hangs on

Shoot days are computed before any dollar figure, and almost every line item is a function of them. The pace is the production scale's pages-per-day — 7 at micro-budget, 5.5 low, 4.5 indie, 3.5 mid-level, 3 studio, 2.2 tentpole — divided into the exact eighths measurement, then multiplied by a *driver load*: one, plus fifteen per cent of the night-scene fraction, plus small increments for stunts, pyro, water and crowds, plus a flat ten per cent when the VFX tier is heavy or above. The floor is five days. Prep is 1.1 times the shoot in weeks (1.6 on crews of 110 or more, minimum two); post is 2.4 times the shoot plus eight weeks for moderate VFX or sixteen for heavy, minimum eight. The week is five days unless you say otherwise; six- and seven-day weeks take their premium from `TMoney.TC_DEFAULTS` in `tools/lib-money.js`, and if that file is absent the premium is reported unavailable rather than quietly applied as 1×.

Cast weeks, drops and pick-ups

Supporting cast are paid for the weeks they are *held*, not the days they work. `castWeeks()` takes the worked shoot days and returns billed weeks, holds and the drop/pick-up arithmetic. The defaults in `CAST_RULES` — five days per week, a minimum ten free days before a drop is permitted at all, one drop per performer, one day of written notice, a one-week minimum guarantee restarting on re-engagement, and `asOfDay: null` meaning nothing has been shot yet — are all parameters, because they are contract terms. A drop whose notice date is already past is *refused* into `refusedDrops` with its reason, and those idle days stay billed as holds, rather than the panel quoting a saving nobody can take.

The tiers, and what each assumes

SELECTOR	DEFAULT	WHAT CHOOSING IT CHANGES
Production scale	Indie ($2M–$10M)	Pages per day, crew size and day rate, art allowance, script rights, producers, sound, music, DI, editorial, casting
Director tier	Emerging	One flat fee range, first-time through legend
Star / lead tier	Rising talent	Flat run-of-picture fee × number of leads
Supporting cast	Scale / indie	The per-role band the weekly cost is clamped into
Crew / unions	Non-union	Fringe rate (28 / 32 / 40 %), a labour multiplier, the performer day and week rates
Locations	Local practical	Per-day fee, per-location permit and restore, travel percentage
Camera & equipment	Indie kit	A weekly rental band, over shoot weeks plus one prep week
VFX intensity	Auto	Shot count and per-shot cost; auto derives both from the VFX and pyro drivers
Genre	Auto	Which benchmark the total is compared against — nothing in the budget itself
Tax incentive	None	Estimated recovery and net cost; twenty-one jurisdictions modelled

Leads are dealt as flat fees because that is how they are actually dealt. Supporting roles are not: their weekly rate — $1,800 non-union, $2,812 on a SAG Low Budget Agreement, $4,326 on SAG Basic — is multiplied by the span weeks the day-out-of-days produced, then clamped into the chosen band. Day players are the day rate ($400 / $810 / $1,246) times worked days.

Lines are grouped Above the line, Production, Post-production and Other on the Movie Magic-style account skeleton. Crew labour splits across department accounts by fixed percentages; equipment splits camera 45 %, grip and electric 50 %, sound 5 %; the art allowance splits 55/30/15 across art, wardrobe, and makeup and hair. Insurance is 2.5 % of direct costs, legal and finance 1.5 %, a completion bond a further 2.5 % on indie-financed scales only, general expenses 4 % of below-the-line, contingency a flat 10 %. The account number prefix on each label is load-bearing — the Producer Suite routes a seeded line by matching it. The headline is three numbers: low, high, and a "likely" that is simply the midpoint. Incentive recovery is total × qualifying-spend fraction × credit rate, with a warning when the estimate falls below a programme's minimum spend or above its cap.

How a producer corrects the model

Inside the panel the corrections are the ten selectors above, plus the retake multiplier and parallelism for the AI figure. The engine accepts more than the panel offers: per-scene breakdown tags (`unitOverrides`) replace the keyword-derived counts for extras, stunt, pyro, water and animal days; real day assignments from the stripboard (`castDood`) replace the script-order day-out-of-days entirely; `castRules` and `daysPerWeek` replace the default agreement terms. Those arrive from the Producer Suite, and the board's numbers win — one implementation of the cast arithmetic serving both surfaces rather than two files disagreeing.

Documentary mode

The panel carries a Feature Film / Documentary switch, and detection is automatic enough to prompt: when interview, archival and narration cues score 60 or more, a scripted estimate offers to switch. Documentary numbers come from `timeline/timeline-doc.js` (`SBDoc`), which returns the same groups-and-totals contract so the rest of the panel works unchanged. Nothing of the scripted model carries over. There are no cast tiers and no stunt units; instead four scales from DIY to premium/streamer, an archival appetite priced in finished minutes at an all-media rate per minute, a music posture, a crew basis, and a shoot of interview days plus vérité days plus one travel day per region. Edit weeks come from weeks-per-finished-hour bands, a shooting ratio is blended from the interview share, and a grants panel lists funding offsets. Incentive eligibility is adjusted where documentary rules are known — New York excludes documentaries outright, Georgia gives the 20 % base without the logo uplift.

> **The switch writes outside this module.** Flipping to Documentary also rewrites the `strategy` field inside the `SB_Sales_v1` storage key so the Sales tab follows, and flipping back resets a documentary strategy to `indie`. Any sales strategy you had chosen by hand is overwritten without a prompt.

An estimate is a model, not a quote

Say this to anyone you hand the panel to. The tables are calibrated against published 2025–26 union scale, permit schedules, rental price lists and trade-press deal reporting, and every figure is sourced by line in `docs/PRODUCTION_PRICING.md`; the documentary constants the same way in `docs/DOCUMENTARY_MODE.md`. Read them before you defend a number. Both publish their own limits, and these bite hardest: marketing and print costs are excluded entirely; fees above "established" are reported figures, not scale; cast fringes are approximated because pension and health caps out on large fees; the day-out-of-days schedules scenes in script order with no stripboard optimisation, so supporting spans run conservative; genre benchmarks are inflation-adjusted 2005-vintage data, order-of-magnitude context rather than forecast; and incentive recovery is modelled on headline terms, where the real number is a production accountant's opinion letter.

The preview half of the panel is simpler and carries the same warning: clip count times average duration times the model's dollars-per-second, multiplied by a retake factor of 1.6 — the average generations per kept clip — with stills at four cents each. A local-render build prices at zero, because there the cost is electricity.

Getting the work back out

`timeline/timeline-export.js` (`SBExport`) produces four things, and only four.

ARTIFACT	FILE	WHAT A RECEIVING APPLICATION DOES WITH IT
CMX-style EDL	`cinamate-timeline.edl`	An online bay, finishing house or NLE conforms the cut: each event carries a source in/out and a record in/out, and the reel names say what media to relink
Project JSON	`cinamate-timeline-project.json`	Cinamate reads it back — clips, characters, location bible, globals, assembly, project name, script text and parse result
Stitched master	`cinamate-final.mp4`	A finished single-file cut of the approved clips, assembled in the browser
Clip archive	`cinamate-final.zip`	The fallback when stitching is unavailable: each clip as `clip_NN.mp4`

The EDL opens `TITLE: CINAMATE TIMELINE`, then `FCM: NON-DROP FRAME`, then a comment stating the frame rate the numbers were counted in — an EDL carries no rate of its own beyond the FCM line, so a conform at the wrong rate is otherwise silent. Events are numbered from 001 and butt together, each record-in the previous record-out, and a zero-length clip is floored to a single frame, because an event whose record-in equals its record-out is one no conform will accept. Duration honours sub-second trims and rounds to frames exactly once.

Exporting from the Timeline sends only clips that are *approved and have video*; if none qualify, it falls back to every clip. The stitcher fetches each approved clip, hands the blobs to `SBFFmpeg.stitchBlobs`, and packages a ZIP through JSZip if that fails. Re-importing a project file goes through `CVault.scrubImported` and is refused outright if that safety layer did not load.

> **Formats this module does not produce.** There is no OTIO, no FCPXML or Premiere XML, and no CSV anywhere in `timeline/timeline-export.js`. If your finishing partner needs one, the EDL and the project JSON are what you convert from.

Why your labels come back without their line breaks

An EDL is line-oriented, and every field is interpolated into it by concatenation. A newline inside a clip label therefore does not produce a long label — it ends the comment and starts a record. A project name of "Heist" followed by a newline and `FCM: DROP FRAME` emits that line above the real one, and CMX readers take the first, reinterpreting the whole reel at the wrong timecode base; a label containing a plausible event line forges a cut that was never made. Every interpolated field therefore passes through `edlField` in `js/safe-url.js` on its way into the file: carriage returns, newlines and the Unicode line separators U+2028 and U+2029 are folded to single spaces, control characters removed, the result trimmed and truncated at 200 characters. The reel identifier is separately synthesised as `CLIP_nn` from the clip number and capped at 32 characters, with the event line's reel column taking the first eight.

> **This is deliberate, not corruption.** A clip label that appears in an exported EDL with its line breaks flattened to spaces, or cut off at 200 characters, is the export doing exactly what it was built to do. Your labels in the project are untouched — only the copy inside the EDL is folded, and the full text still lives in the project JSON.

The frame rate

Every conversion in the exporter passes through `normFps`, so an absent, zero, negative or nonsensical rate becomes 24 in one place rather than differently at each call site. An unstated rate is therefore not treated as unknown — it is stamped as 24 with total confidence, and a 30 fps master turned over at 24 lands 25 per cent long with no error anywhere.

> **A gap this manual cannot close by reading.** The Export panel offers a Master frame rate picker (`tlFps`, 23.976 through 30). No code in the repository reads or writes that element, the `fps` field on the Timeline state is initialised to 24 and never read again, and both export calls in `timeline/timeline.js` are made without a rate argument — so on the source as it stands, both files are stamped at 24 whatever the picker shows. Read the `* FRAME RATE:` line in the export before you send it to a conform.

Two Node suites assert this in isolation. `scripts/test_timeline_export.mjs` checks the timecode arithmetic at real cut lengths — an hour, ninety minutes, sub-second trims — because under a minute a wrong hours field

looks like a right one. `scripts/test_budget_engines.mjs` enforces that one file defines `SBBudget` and that every page loading it also loads `timeline-doc.js`, so a documentary cannot be priced differently on two screens.

The Producer Suite I — The Budget Top Sheet

A top sheet is a promise about money made in public. This chapter describes exactly how Cinamate arrives at every figure on it — which numbers you type, which the sheet derives, and the order the derivations run in, because that order moves the grand total.

> **On the accuracy of this chapter.** Everything below was established by reading the source files named in it. The platform is being worked on continuously; if a figure on your screen contradicts a sentence here, the running software is the authority and this page is out of date.

Three levels, one stored sheet

The Budget tab of the Producer Suite (`producer/index.html`, driven by `producer/budget-sheet.js`) is a three-level hierarchy: the *top sheet* on the left, a list of account categories; the *detail table* on the right, the line items inside whichever category is selected; and beneath both, the derived totals. Selecting a row on the left changes what the right-hand table edits.

There is exactly one sheet. It autosaves to browser storage under the key `SB_BudgetSheet_v1`, on a 300 ms debounce after any edit. There is no version history and no undo — a replaced sheet is gone unless you exported it first.

The chart of accounts

Account numbers here are not local to this page. They come from `js/lib-money-accounts.js`, the single chart every module that posts money shares: crew deal memos, the casting office, VFX bids and the Money Room cost report all commit against these same numbers, which is what lets an actual find its way back to the budget line that anticipated it.

A new sheet carries nineteen major accounts. Eleven are *labour* accounts, and that flag defines the base payroll fringes are computed on.

ACCT	CATEGORY	LABOUR	ACCT	CATEGORY	LABOUR
1000	Story & Rights		11000	Makeup & Hair	yes
2000	Producers Unit	yes	12000	Transportation	yes
3000	Direction	yes	13000	Locations	
4000	Cast	yes	14000	Media & Stock	
5000	Production Staff	yes	15000	Post-Production	
6000	Camera	yes	16000	Insurance & Legal	
7000	Sound	yes	17000	Publicity	
8000	Grip & Electric	yes	18000	General Expenses	
9000	Art Department	yes	20000	Payroll Fringes	
10000	Wardrobe	yes	19000	Contingency	computed

Two absences are deliberate. **19000 Contingency** is not a category you can type into — it is derived from everything else, so seeding explicitly discards any contingency line offered to it. **20000 Payroll Fringes** exists as its own account precisely because fringes are cross-departmental; folded into General Expenses they read as several times the whole of an account whose job is to be a small percentage of below-the-line.

The chart also carries *detail* accounts that departments post to, each rolling up to exactly one major above: 4100/4200/4400/4500 under Cast, 8500 under Grip & Electric, 9100 and 9900 under Art Department, 13500 under Locations, 15200/15400/15600/15800 under Post-Production, 16500/16800 under Insurance & Legal. Anything the chart does not recognise rolls up to itself, so it surfaces on the cost report as `Unbudgeted` rather than disappearing into a real line.

Line items — Amt × Units × Rate

A line item stores a description, three calculator fields (Amt, Units, Rate), an Estimated figure, an Actual figure and free-text notes. The rule deciding what a line is worth lives once, in `js/lib-money-sheet.js`:

- If **all three** of Amt, Units and Rate are greater than zero, the line is worth Amt × Units × Rate, and the Estimated cell becomes a read-only computed display.
- Otherwise the line is worth whatever is typed into Estimated, which stays an editable field. This is the lump-sum path.

A zero or a negative in any of the three calculator fields therefore switches the line back to the manual figure. Numbers are read tolerantly — `1234.56`, `$1,234.56` and `1 234.56` all parse the same, and accounting parentheses `(50)` read as minus fifty. Anything genuinely unreadable reads as zero; a budget cell is never allowed to become `NaN`.

On save, the computed figure is written back onto the line item's stored Estimated value. That is not cosmetic: five readers across finance, the investor letter and the learning layer sum the stored value alone, so before this a fully built sheet of calculator lines read as zero everywhere outside this page.

Category subtotals and the percentage column

A category subtotal is the sum of its line items' values, with an Actual subtotal alongside it summed the same way. The percentage at the right of each top-sheet row is that category's share of the *grand total* — not of the direct subtotal — rounded to a whole percent.

The derived layers, and the order they run in

Four figures are computed rather than typed. The order matters, and it is fixed in `sheetTotals()`:

1. **Direct subtotal** — every line item in every category.
2. **Payroll fringes** — the fringes percentage applied to the *labour base*, which is the subtotal of the eleven labour accounts only. Not the whole sheet.
3. **Completion bond** — the bond percentage applied to the direct subtotal.
4. **Insurance** — the insurance percentage applied to the direct subtotal.
5. **Contingency** — applied last, to the sum of the four figures above. Contingency therefore covers the fringes, the bond and the insurance, not merely the direct costs.

Bond and insurance quote on the direct subtotal and nothing else, so raising the fringes percentage does not move either of them. Worked through on a round million, with a labour base of $400,000:

LAYER	BASE	RATE	AMOUNT
Direct subtotal	—	—	1,000,000.00
labour base (a subset, not added again)	—	—	*400,000.00*
Payroll fringes	labour base	28%	112,000.00
Completion bond	direct subtotal	2%	20,000.00
Insurance	direct subtotal	2.5%	25,000.00
Contingency basis	sum of the four	—	1,157,000.00
Contingency	basis	10%	115,700.00
Grand total			**1,272,700.00**

Had contingency been taken on the direct subtotal instead — the order most hand-built spreadsheets use — it would be $100,000 and the grand total $1,257,000. The same inputs, $15,700 apart. Before arguing about someone else's top sheet, establish which order produced it.

All four percentages are edited in the header strip above the top sheet, each clamped: contingency 0–30%, fringes 0–45%, bond 0–6%, insurance 0–8%. A value outside the range is pulled back to the nearest end rather than rejected.

> **Fringes default to zero, on purpose.** A new sheet's fringes, bond and insurance percentages are all zero, while the default skeleton already contains a *Cast fringes* line under 4000 and a *Payroll fringes (labor accounts)* line under 20000. Fill in those line items or set the percentage — doing both counts your fringes twice, and nothing warns you.

Why every figure is held in cents

All arithmetic on this sheet runs through `js/lib-money-math.js`, which carries money as an integer number of cents. Rounding happens once, half away from zero, at the moment a fraction of a cent is created — a multiplication or a percentage — and never on a sum. A total is then the sum of the cents that were actually posted, so it foots by construction rather than by luck.

This is not fastidiousness. A JavaScript number is a binary float: 3 × 5 × 1061.64 is 15,924.6, and left as a float it prints as `15924.599999999999`. Rounding each row before adding, rather than adding what was posted, produced a cost report whose TOTAL row disagreed with its own columns by tens of dollars across a couple of hundred accounts — on the report that goes to the completion bond.

Money therefore leaves the platform as a fixed two-decimal *string*, not as a raw number, and that is what makes an exported column safe to sum. `scripts/test_ops_money.mjs` asserts it: every money cell in an exported report must match `-?\d+\.\d{2}`, with no cell anywhere carrying three or more decimals.

Actuals against estimates

Every line item carries an Actual beside its estimate, and category and sheet actuals are summed from those. When the sheet holds any actual, the grand total row prints it alongside the estimate.

If the Money Room has posted anything — purchase orders, petty cash or payroll in `SB_Money_v1` — the top sheet also prints a one-line cost summary beneath the totals: estimated final cost, and the variance marked *over* or *under*. It is computed by the Money Room's own cost report, not recomputed here.

Actuals also feed the estimator's learning layer (`js/learn.js`, stored under `CIN_Learn_v1`), which learns a per-account correction from lines carrying both an estimate and a real actual. It is gated: a production teaches once, when marked wrapped in `CIN_Projects_v1`. Mid-shoot, a department that has spent 12% of its budget in week two looks like an 88% underrun, and folding that in would drag every future estimate down.

Cash needed by phase

Below the grand total the sheet prints a three-way split — prep, shoot, post — of when the money leaves the bank. Each account carries a fixed phase profile (rights entirely prep, post-production almost entirely post, cast overwhelmingly shoot); the derived layers are split between shoot and post, the last bucket absorbing the remainder so the three figures add back to the grand total exactly.

The norms advisor

Each top-sheet row is measured against a band of typical shares of the grand total for that account, held in the `NORMS` table in `producer/budget-sheet.js`. A category outside its band gets a marker — ▲ heavy, ▽ light — with the band in the marker's tooltip, and a line beneath the totals counts how many accounts are flagged. Being outside a band is not an error. It is the question the bond company will ask, asked early.

Seeding from the script estimate

Seed from script estimate runs the script-driven estimator over the current project, routes every estimated line to the top-sheet category its account rolls up to, and writes the **midpoint** of each low–high range into the line, preserving the range in that line's notes. Where the learning layer holds at least two past actuals on an account and a correction other than 1.0, the seeded figure is multiplied by it and the note records by how much. Contingency lines are dropped, since contingency is derived. Where the Schedule tab's stripboard carries real day assignments the estimator uses them, and the confirmation reports what the cast drops saved in weeks.

> **Seeding replaces the whole sheet.** Every line item in every category is deleted before the estimated lines are written, and the contingency percentage is reset to 10%. Anything you had typed by hand is lost, including actuals. Export first — *Save (.json)* — if the sheet holds work you cannot retype.

The CSV export

Export CSV downloads the sheet as `<sheet name>.csv`, the name stripped of anything outside letters, digits, hyphen and space. Columns: `Account`, `Category`, `Line item`, `Amt`, `Units`, `Rate`, `Estimated`, `Actual`, `Notes`. Each category writes one row per line item followed by a `SUBTOTAL` row when it holds anything. At the foot: `SUBTOTAL (all categories)`; a fringes, bond and insurance row each, but only when that percentage produced a figure; a contingency row always; and `GRAND TOTAL`. The Actual column is left empty rather than written as `0.00` when there is nothing in it.

Estimated and Actual are written as fixed two-decimal strings, for the reason given above; a spreadsheet reads them as numbers and sums them correctly.

The leading apostrophe is deliberate

A cell whose first character is `=`, `+`, `-`, `@`, a tab or a carriage return is treated by Excel and Google Sheets as a *formula*, not as text. A line item description typed into this sheet would then execute on the machine of whoever opened the export. The exporter therefore prefixes such a cell with a single apostrophe, which both spreadsheets read as "the rest of this cell is text".

> **Do not strip those apostrophes.** If you see a cell reading `'=Second unit day`, that is the neutralisation working as designed. One consequence to expect: a genuinely negative money figure exports as `'-1234.56`, because it too begins with a hyphen, and a spreadsheet will treat it as text rather than as a number. If you need a credit to behave arithmetically in Excel, re-enter that one cell after opening the file.

Saving, loading, printing, starting over

Save (.json) writes the complete sheet — categories, line items, percentages, actuals — to `<sheet name>.budget.json`. That file is the archive copy; the CSV is a report, and cannot be loaded back. *Print / PDF* hands the rendered sheet to the browser's print dialogue.

> **Load and New sheet both replace the live sheet.** *Load* accepts a `.json` file and, if it parses and contains a categories array, swaps it in immediately — with no confirmation prompt. *New sheet* asks first ("Start a new blank top sheet? The current sheet will be replaced (save it first if needed).") and then discards everything. Both write straight to `SB_BudgetSheet_v1`. There is no undo.

What this sheet is not

Stated plainly, because a document full of account numbers invites the assumption:

- **It is not an accounting system.** One estimate and one actual per line, with no ledger, no double entry, no reconciliation and no audit trail. The Money Room tracks commitments and spend; neither is a set of books.
- **It files nothing.** Nothing here is submitted to a jurisdiction, a payroll company, a guild, a bond company or a tax authority. Every export is a file that lands in your downloads folder.
- **The incentive figures are estimates from a table.** The Incentives tab computes recovery as budget × qualified-spend percentage × credit rate against a stored table of headline program terms, and flags a budget below a program's minimum spend or above its cap. Real programs carry qualification rules, allocation queues, sunset dates and audits that no table models. Treat the output as a shortlist for a production accountant, never as advice.
- **The on-screen figures are abbreviated.** The top sheet shortens money — a grand total of $1,272,700 reads as `$1.3M`, a line of $15,924.60 as `$15.9k` — and clamps a negative to zero for display. The exact figures live in the CSV and the JSON. Quote from the export, not from the screen.

The Producer Suite II — Scheduling, DOOD & Call Sheets

A broken-down script is a list of scenes. A schedule is a claim about how many days those scenes will take, who is owed money on which of them, and what time a hundred people arrive tomorrow. The Schedule tab makes all three from the same board — and it will tell you, in as many words, when the first of them is still a guess.

> **How this chapter was written.** By reading the platform's source, not by driving it. Cinamate is under active development. Where a screen and a paragraph here disagree, believe the screen.

The stripboard

The board lives in the Schedule tab of the Producer Suite and is stored under one key, `SB_ScheduleBoard_v1`. Everything below writes there.

Seed scenes from script builds the strips. Each slugline in the timeline's script becomes one strip, sized in eighths of a page at five script lines to the eighth (`LINES_PER_EIGHTH` in `timeline/timeline-budget.js`), with a one-eighth floor. A strip is marked *night* when its slugline contains NIGHT, DUSK, EVENING, MIDNIGHT or PRE-DAWN, and *day* otherwise. Cast comes from the estimator's per-scene map — a ranked character name occurring in that scene's text. New strips land on day `-1`: the boneyard.

> **Seeding replaces the board.** Seed scenes from script discards the existing strips outright and asks nothing first. Breakdown tags, extras counts, scene notes, hand-edited page counts and every day assignment go with them. Re-seed when the script has genuinely changed, not to refresh the view.

Double-clicking a strip opens its breakdown: slugline, page count, day or night, background count, cast, notes, and six tags — stunts, SFX, VFX, water, animals, vehicles. The page field accepts what an AD actually writes: `1 7/8`, `7/8`, `15`, or a decimal like `2.5`. Deleting a scene from that panel asks first.

Strips are ordered two ways. Drag them between the boneyard and any day — there is always one empty trailing day to drop into. Or press **Auto-schedule**, which fills days at the target pace in *script order*, or in *group by location* order, which sorts by the set name parsed out of the slugline and puts day work before night work at each location: fewer company moves, the way a board is really laid out. Scene numbers are preserved either way.

> **Unschedule all** returns every strip on the board to the boneyard immediately, with no confirmation and no undo. The strips and their breakdowns survive; the day assignments do not.

Shoot days are keyed on date *and* unit

A day on the board is an index. A day on set is a date. The record that joins them is `SB_ShootDays_v1`, in `js/lib-shootdays.js`, and it holds exactly this: `dayIdx`, `date` as `YYYY-MM-DD`, `unit`, the `sceneIds` scheduled that day, `wrapped`, and `pinned`.

The identity is the pair — date and unit, never the date alone. Main unit and a splinter unit can both shoot one date; that is two days' work, two call sheets and two rows on the daily report. Lookups therefore take a unit alongside the date, and `unitsOnDate` lists every unit working a given date. Asked without a unit, the lookup answers with the first record on that date, which is the main unit because indices sort ascending. The default unit is `MAIN`.

Dates come from the **Day planner**, the bar `tools/sched-weather.js` injects above the board. It takes day 1's date, a city or a latitude and longitude, a skip-weekends switch and an optional day count, and saves them to `SB_ShootPlan_v1`. From there, day *n* is *n* shoot days forward of day 1, stepping over Saturdays and Sundays unless skip-weekends is off, and computed at noon UTC so a browser in any timezone lands on the same calendar day.

Pinning is the exception to that arithmetic. A record is pinned when a writer *stated* its date and unit rather than deriving them — which is what happens when Dailies names the day it is shooting. A rebuild recomputes the derived halves of every record and leaves the pinned halves alone, so a take dated to a day the stripboard has never heard of still resolves.

The pace, and why it stays a guess until you wrap a day

The board ships with a planning pace of **4.5 pages per day**. It is a default for a film nobody has shot yet, and it is deliberately the only place that number appears.

The board can do better than that: it can learn the pace this production actually shoots. But it learns from one thing only — **shoot days an operator has explicitly marked wrapped**. Until three such days exist, the board reports the shipped default and says so in plain words on the chip beside the toolbar:

```
target 4.5 pg/day — the shipped default; nothing learned yet (no wrapped days)
target 4.5 pg/day — the shipped default; 2 wrapped days so far, 3 needed to learn
target 3.25 pg/day — learned from 4 wrapped days
```

An operator who never wraps a day will read the first of those lines for the whole shoot, and every day count on the board — and every number downstream of it — will rest on a figure nobody measured.

How to wrap a day

The control is on **Today**, the on-set page, not on the board. Beside the day selector is a button reading ◉ **Wrap day**. Pick the day, press it, and confirm the prompt; the button then reads ↺ **Re-open day** and reverses the same flag. That is the only shipped control that writes `wrapped`, and it is deliberately on the page the people standing on the day are looking at: whether a day with no takes logged against it is an unshot day or a wrapped one is something only they know.

The board shows the result but does not set it. A wrapped day carries a small read-only WRAPPED chip in its header on the stripboard, and the same word appears on that day's call sheet. There is no toggle there.

What is learned, exactly

For every wrapped day, in index order, the board compares two numbers: the eighths *planned* on it from the strips, and the eighths it *achieved*, read from the take log. Takes are gathered from both stores that hold them — `SB_TakeLog_v1` and `SB_Dailies_v1` — and matched to the day by its calendar date. Each take names a scene; that scene's eighths are counted **once for the production**, however many takes it took and however many days it was touched on. A day that re-shoots a scene already in the can reports it as a pick-up worth zero new pages, because the pace being measured answers "how many days to cover the script".

The learned pace is the **median** of the achieved column, divided by eight, clamped between 1 and 12 pages. A median, not a mean, because a single catastrophic day should not move the target on its own — and three is the smallest count at which the median is a real order statistic. A wrapped day whose take log is empty is not evidence at all: reading that as zero pages would teach the board that a production which does not log takes shoots nothing.

A target you type always wins. Setting the Target pages/day field marks the pace as yours and the board schedules at it, while still reporting what your wrapped days achieved next to it. **Use learned pace** gives the override back.

> **One caveat on un-wrapping, established by reading.** Re-opening a day reverses the flag in `SB_ShootDays_v1`, and the pace model recomputed from scratch drops that day's evidence — this is proved by the suites. The board, however, keeps its own merged pace log inside `SB_ScheduleBoard_v1`, and the merge can only retract a row when the caller tells it which day indices it examined. The board's single call site (`producer/schedule-board.js`, in `syncLearning`) does not pass that argument, so a row written while a day was wrapped appears to survive the un-wrap in the stored log. Treat wrapping as a statement of fact rather than a switch to try.

Day-out-of-Days

The DOOD button opens a grid: one row per cast member, one column per shoot day, sorted by days worked. The letters **compose** rather than branch, so a cell can say everything true of that day at once:

CODE	MEANING
S	Start — the performer's first day
P	Pick-up — re-engaged after a drop
W	Work
D	Drop — released after this day
F	Finish — the last day of the engagement
H	Hold — an idle day inside an engagement, still billed
—	Dropped — an idle day the production released, not billed

So SW, WF and SWF read as they always did, and SWD ("starts, works and is dropped") or PWF ("picked up, works, finishes") can be said at all. The trailing columns are TOT span, WRK worked, HLD billed holds, DRP dropped days and WKS weeks billed, with the weeks a drop saved shown against the row.

Which idle stretches qualify as a drop is not this board's decision. It calls `SBBudget.castWeeks` in `timeline/timeline-budget.js`, the single implementation of that arithmetic on the platform, so the letters on

the grid and the money on the budget cannot disagree. Its shipped terms: ten free days minimum before a gap is droppable, one drop per performer, one day's written notice, a one-week minimum per segment. Notice falling before day 1 is served at engagement and the drop stands; notice falling on a day already shot is refused, and those idle days stay billed as holds rather than quoting a saving nobody can legally take.

The **working week** selector accepts 5, 6 or 7 days. It changes the calendar — the same days in fewer weeks — and the money: the sixth- and seventh-day premiums are read from the timecard engine's defaults, so a six-day week averages (5 + 1.5) ÷ 6 per crew day. If the money library is not loaded on the page, the premium is reported as unavailable rather than quietly applied as 1×.

Where day counts reach the estimate

When the Budget tab seeds from the script estimate, it asks the board for overrides first. A scheduled board hands back exact cast spans per performer, the working week, and special-unit day counts read off the breakdown tags — stunt, pyro, water and animal days, plus a background total — which override the estimator's keyword heuristics and its script-order guess at the DOOD. An unscheduled board hands back nothing, and the estimator falls back to its own model: page count over a scale pace, multiplied by a difficulty load, with a five-day floor. That fallback pace is the estimator's own, not the board's learned one.

The call sheet

Every day header carries a clipboard button, and it opens that day's sheet. Almost nothing on it is typed: the sheet is assembled from fields that already existed one hop away.

BLOCK	COMES FROM
Date, unit, wrapped	`SB_ShootDays_v1`
Scenes, pages, cast, tags	the strips on that day
Sunrise, sunset, golden hours	computed locally from the pin on `SB_ShootPlan_v1`, at the location's civil offset
Sets, addresses, parking, load-in, hospital	`SB_ScoutBook_v1`
Department calls, walkie channels	`SB_Crew_v1`
Meals, wrap, turnaround	the timecard engine's defaults
Advance schedule	the next two days on the board

The whole clock derives from one number you enter: the general crew call. Shooting call is 30 minutes after it, breakfast 30 before it, lunch six hours from call for 30 minutes, second meal six hours after that, estimated wrap on the twelve-hour convention plus the meal, and the earliest next call ten hours after wrap — the same rules the timecard engine bills against, so the sheet cannot promise a meal that payroll then penalises. Departments offset from the crew call: art and hair-and-make-up an hour early, wardrobe 45 minutes, grip-and-electric and production 30, camera and sound with the unit. Walkie channels go only to departments this show actually crews, production on channel 1.

Individual cast calls come from the board: the eighths scheduled before a performer's first scene that day, pro-rated across the shooting window, backed off an hour for hair, make-up and wardrobe, rounded to five minutes. A

per-name override stored on the day is honoured if present, though no shipped control writes one. Each DOOD letter is spelled out — START, WORK, PICK-UP, HOLD, DROP AFTER TODAY, FINISH — because the sheet goes to everyone, not only to an AD, and a drop day carries the date the notice is due.

Anything the sheet was not handed is listed at the foot under *Not on this sheet, and why*, naming the store to fix: no crew directory, no hospital on file, no location pin, no call time. The forecast is the one line that cannot be computed on the machine; it is fetched live, and a blocked or failed request says so rather than impersonating clear skies.

Revisions are derived, not remembered. The sheet's content is reduced to a signature; re-printing an unchanged sheet is not a revision, while a sheet that has moved since its last issue is labelled REV. A PENDING until it is re-issued. Issue / print stamps the issue and prints; issues are kept on the day and capped at 26.

Getting it off the screen

Print / PDF on the toolbar opens the Day-out-of-Days and prints the board with it; the call sheet has its own Issue / print. There is *no* CSV export on the Schedule tab. The Export CSV button in the Producer Suite belongs to the Budget tab and exports the top sheet, not the schedule — print to PDF is the schedule's export path.

What this scheduler does not do

- It does not know about cast availability it was not told about. Nothing on the board says a performer is on another film in week three.
- It does not book, hire, hold or notify anyone. A drop notice date is arithmetic; serving the notice is a person's job.
- It does not send a call sheet. It draws one and prints it.
- It does not enforce a turnaround, a meal or a sixth day — it flags a short turnaround and quotes the rule it measured against.
- It does not know the weather. It quotes a forecast, or says it could not get one.

A generated call sheet is a **draft for a human first AD to check**. It is assembled correctly from what the platform holds, which is a different claim from being right about tomorrow.

What is guaranteed

Two suites pin this chapter, both running offline against the libraries. `scripts/test_shootdays.mjs` holds 95 assertions on the shoot-day record: the weekend-skipping calendar, the date-and-unit key, two units sharing one date, pinning surviving a rebuild, wrapping and un-wrapping through the store. `scripts/test_schedule_learn.mjs` holds 145 on the schedule: that three takes of one scene count its pages once, that only wrapped days become evidence, that the learned pace is the median and not the mean, that a typed pace overrules a learned one, and that every block of the call sheet is what it claims to be.

Set Design & The 3D Builder

A set plan is two drawings that must never disagree: the one the art department builds from, and the one the DP frames against. The Set Designer keeps them as a single record — the same feet, the same rotations, the same format — and stands one up from the other with no conversion step between.

> **How this chapter was produced.** Every statement below was established by reading the files it names, not by operating the application. Cinamate is under continuous development; where a sentence here and the running software disagree, believe the software and treat this page as stale.

What this is, and what it is not

The Set Designer at `sets/index.html` draws a stage in feet, places flats and dressing on it, puts real lenses at real marks, and stands the result up so a designer can see whether a doorway reads and a DP can see what a lens covers. It is not a CAD package: there is no dimension chain, no wall assembly, no tolerance and no structural calculation in `sets/lib-set.js` or `sets/lib-set3d.js`. A flat is a rectangular prism with a height; it has no studs. These are planning measurements, not construction drawings, and nothing leaving this module has been engineered.

The plan, and what the units are

Everything is measured in **feet** — the plan, the 3D geometry, both model exports. Neither module works in metres or in pixels; the only metric step is internal to the depth-of-field maths, which converts back before the number reaches you.

A new document opens with one plan, *Set 1 — Untitled*, 24 ft by 18 ft, editable in the right-hand inspector. The stored value is floored at 6 ft; the field advertises a 200 ft ceiling, but that belongs to the form control rather than the model, so a larger value typed past it is kept.

Snapping is a **half-foot grid** — a six-inch module — applied on every drop and drag. Dragging clamps an item's *centre* to the stage rectangle, so a ten-foot flat parked on the edge overhangs by five feet, which is what a wall on the edge of a stage does. Each plan carries a free-text *Scenes* field, tying the set to the scenes it serves.

The stencil catalog

Nineteen stencils are defined, each with a footprint in feet and a matching 3D profile giving it a height and a shape. Clicking a palette button drops the piece mid-stage; you then drag it into place.

PIECE	PLAN W×D FT	HEIGHT FT	3D SHAPE	PIECE	PLAN W×D FT	HEIGHT FT	3D SHAPE
Wall / flat	10 × 0.5	10	box	Rug	8 × 5	0.05	box
Door	3 × 0.5	6.75	opening	Plant	1.5 × 1.5	4	cylinder
Window	4 × 0.4	4, at 3 off floor	opening	Piano	5 × 6	3.3	box
Table	5 × 3	2.5	top + legs	Vehicle	15 × 6	5	box
Chair	1.6 × 1.6	3	seat + back	Green screen	12 × 0.5	12	box
Sofa	7 × 3	2.7	base, back, arms	Blocking mark	1.2 × 1.2	5.8	figure
Bed	6.5 × 5	2	box	Camera	1.6 × 1.6	5	body + lens
Desk	5 × 2.5	2.5	top + legs	Light	1.4 × 1.4	7	head + stand
Counter	8 × 2	3	box	Custom box	4 × 4	3	box
Shelf	3 × 1	6	box				

Every number there is a default. The inspector exposes W, D, rotation, *Height ft*, *Off floor ft* and a colour on any selected piece. A nonsensical height falls back to the profile default rather than producing a broken mesh, and an unrecognised stencil type resolves to *Custom box*.

Walls, flats and openings

A wall is a box, ten feet tall by default. Doors and windows are not solids: each is built as the frame *around* a hole — two jambs three-tenths of a foot wide, a header above the opening, a sill below it where the piece sits off the floor. A 6.75 ft door in a ten-foot wall gets a header and no sill; a 4 ft window raised 3 ft gets both. Those frames run to the tallest wall on the plan, or ten feet if that is taller, so raising one flat to sixteen feet raises every door frame with it.

Camera and lens

A Camera stencil carries a focal length in millimetres (35 mm by default) and an aperture (f/2.8), and the plan itself carries the **format** it is shot on. That one key answers every optical question the module asks — the cone on the plan, the frustum in the 3D view, the frame you look through — so the plan and the viewport cannot report different lenses. The format table lives once, in `tools/lib-media.js` as `TMedia.SENSORS`, and both set modules read it rather than keeping copies. Nine formats are offered; the default is `super35`.

KEY	FORMAT	MM	FRAME ASPECT	35 MM COVERS
super35	Super 35	24.9 × 18.7	1.33	39.2°
super35-17x9	Super 35 17:9	24.6 × 13.1	1.88	38.7°
fullframe	Full frame	36 × 24	1.50	54.4°
alexa-lf	ARRI LF	36.7 × 25.5	1.44	55.3°
alexa-65	ARRI 65	54.1 × 25.6	2.11	75.4°
red-vv	RED V-RAPTOR VV	40.96 × 21.6	1.90	60.7°
mft	Micro 4/3	17.3 × 13	1.33	27.8°
s16	Super 16	12.52 × 7.41	1.69	20.3°
iphone-main	Phone main cam	9.8 × 7.3	1.34	15.9°

Field of view is plain thin-lens trigonometry: twice the arctangent of half the sensor dimension over the focal length — horizontally against the sensor's width, vertically against its height. The inspector prints both to one decimal beside the format's name, because a coverage figure with no format stated next to it is a number nobody can check.

The frame's *shape* comes from the format and only from the format. In the 3D view the lens frame is letterboxed inside whatever shape the panel is; widening the browser window adds matte, never coverage.

> **A gap worth knowing about.** The frame aspect is the sensor's physical aspect. Nothing in these files implements an anamorphic squeeze or a delivery-aspect crop, so a 2.39:1 extraction cannot be dialled in — choose the format closest in shape and read the safe-area guides described below.

Depth of field

Depth of field is computed in feet. The circle of confusion is the frame diagonal divided by 1500 — the convention yielding roughly 0.029 mm on full frame and 0.021 mm on Super 35. Hyperfocal distance follows from focal length, aperture and that circle; at or past it the far limit is reported as infinity rather than as a large fiction.

What the camera is *focused on* defaults to the answer a 1st AC would give: the nearest Blocking mark the camera is pointed at — within sixty degrees of the lens axis, at least half a foot away, in front rather than behind. With no mark placed it falls back to 12 ft, and any camera may override it outright.

On the plan the sharp band is two arcs struck across the camera's cone, clamped at 60 ft so a deep stop does not run off the stage; the inspector prints the same limits in feet and inches. The 3D view reports the band in its caption but does not blur — there is no defocus rendering in the viewport.

> **Where the numbers stop.** Depth of field requires `tools/lib-media.js`. The Set Designer loads it; a page that loads the set geometry alone — the Props module does — gets fields of view but no sharp band, and the arcs simply do not appear.

Lighting

A Light stencil draws a throw on the plan: a fixed wedge of about forty degrees, nine feet long, aimed by the piece's rotation. It is a placement indicator, not a photometric model — there is no intensity, colour temperature, falloff or beam-angle field in the source. The distinction carries into three dimensions, where the viewport is lit by two fixed directional lights baked into its shader — a key from high front-left, a cooler fill from behind, plus ambient — chosen so every face stays distinguishable. Placing a Light puts a 7 ft stand-and-head model in the scene; it illuminates nothing, and the viewport casts no shadows.

The 3D builder

The 3D view is a second way of looking at the same items: nothing is converted and nothing duplicated, so an edit in one appears in the other immediately. Furniture is built as furniture rather than as crates — a table is a top on four legs, a chair adds a back, a sofa is a base with a back and two arms — which is what makes a plan legible at a glance.

The conventions are worth stating plainly, since they govern what an export looks like elsewhere:

- Units are feet.
- World axes are X right, **Y up**, Z toward the viewer — right-handed, matching OpenGL. A plan point (x, y) becomes world (x, 0, y), so the plan's depth axis is the world Z axis.
- An item's rotation is degrees clockwise on the plan, which is a rotation about the world Y axis, matching SVG's own `rotate()`. A camera at rotation 0 points up the page, which is −Z.
- Triangles wind **counter-clockwise seen from outside**, so a face normal points away from the solid it belongs to. Back-face culling is enabled in the renderer, which makes a winding mistake show up as a missing wall rather than as nothing at all.

The viewport draws a floor grid at one line per foot, brighter every ten, with the stage outline picked out, and draws every camera's frustum as wireframe so coverage reads without leaving free orbit. Drag orbits, shift-drag or right-drag pans, scroll dollies, a pinch zooms; clicking selects the same item a click on the plan would.

Looking through the lens

Look through puts the viewport at a camera's mark, at that camera's real lens on the plan's format — the thing a general-purpose modeller cannot do, because it has no idea what a 35 mm on Super 35 covers. While locked, orbit and zoom input is refused, the image is letterboxed to the format's aspect, and the frame carries the guides every monitor on the floor has: the frame edge, a 90% action-safe box, an 80% title-safe box and a centre cross. The caption names the camera, focal length, format, horizontal and vertical coverage, aperture and sharp band. Delete that camera, or switch plans, and the lock releases itself and says so rather than claiming to be a lens it no longer has.

> **If WebGL will not start.** The 3D panel reports that it is unavailable and the module carries on. The plan view and every export still work; nothing about the plan depends on the viewport.

Exports, and what each is for

Four buttons, all producing downloads named after the plan, lower-cased, with runs of non-alphanumeric characters becoming hyphens.

SVG — the plan

A vector top-down plan at 14 pixels to the foot, carrying the grid, every piece, camera cones with their depth-of-field arcs, light throws, the plan name and dimensions, the format's name, and a labelled 5 ft scale bar. It opens in Illustrator, Figma or a browser.

This file is **self-contained by design**. Because it is downloaded and opened elsewhere, every fill and stroke is written as a literal colour rather than inherited from the application's theme — no stylesheet travels with it and there is no theme to inherit from. That includes the background, which is the same deep navy the application uses, so sending the file straight to a printer lays down a full-bleed dark field; a light plan for paper wants recolouring in your editor rather than a setting here. The export also never carries your current selection highlight.

PNG — the call sheet

The same plan, rasterised through a canvas at twice that size, for dropping into a call sheet or an email.

OBJ — the modeller

Plain text, in feet, with the units stated in a header comment, and one named group per piece. Faces carry three or four one-based indices, and each face declares its own corners: vertices are *not* shared between faces, so a six-sided box exports twenty-four vertices rather than eight. An importer will therefore show every edge as hard — weld or merge-by-distance for a smooth mesh. There is no material library and no `usemtl`, so the per-item colours set in the inspector live in the viewport only and do not survive the export.

STL — previz and printing

ASCII STL, one solid, every facet a triangle with its normal declared explicitly. Facets with no area are dropped rather than written, because a zero-length normal is not a direction and a slicer will believe whatever it is told.

> **STL declares no units.** The format has no mechanism to. The numbers written are feet, and most slicers read STL as millimetres, so a ten-foot flat arrives as a ten-millimetre one. Scale on import — by 304.8 if the receiving tool works in millimetres.

Names are filtered on the way out

Every name written into an OBJ or an STL passes through a slug filter: any run of characters outside `A–Z a–z 0–9 _ -` becomes a single underscore, and the result is truncated to 60 characters. A set called *Kitchen — north wall* is written into the file as `Kitchen_north_wall`. This is not cosmetic tidying — a newline in a name would otherwise end an OBJ comment and let the rest be read as geometry.

One thing to expect rather than discover: an OBJ group name is built from the piece's *internal identifier and its stencil type* — something of the form `i7k2m4a_wall` — not from the label typed in the inspector. Labels appear on the plan and in the SVG; they do not reach the model exports at all. If the receiving artist needs to know which flat is which, the SVG plan is the document that tells them.

What a receiving modeller should expect

Both exports wind counter-clockwise from outside, so face normals point outward and a default single-sided material renders correctly with back-face culling on. The suite pins that rather than trusting it: a box top is measured as pointing exactly +Y and its bottom exactly −Y, the STL is checked to declare `facet normal 0 1 0` and `facet normal 0 -1 0` for them, and every shape in the catalog is checked by the divergence theorem to enclose a positive volume, which an inside-out mesh cannot. Degenerate geometry is refused at source too: faces repeating a corner are cleaned before writing, cap fans go out as genuine triangles rather than quads with a doubled corner, and cylinders are capped at both ends, so a shape arrives as a closed solid a slicer will accept.

Where plans live

The document — every plan, every piece — is stored in this browser under the key `SB_SetDesign_v1`, written after each edit. It is local to the browser and the machine, and there is no version history.

> **Deletions are immediate and final.** Deleting a set plan asks for confirmation once, then removes it and everything on it. Removing a piece — by the inspector's button, or by Delete or Backspace while a piece is selected and no text field has focus — asks nothing at all and cannot be undone. Export anything you would mind losing before clearing a browser's site data, which takes the whole document with it.

What the suites hold in place

Two node suites cover this module without a browser: `scripts/test_set.mjs`, 69 assertions for the plan model, snapping, fields of view, depth of field and the SVG; and `scripts/test_set3d.mjs`, 132 for geometry, matrices, orbit, picking, winding and both exports. The one worth knowing as an operator is the agreement assertion: the plan, the 3D frustum and the shared format table are each asked the same lens question at 18, 25, 35, 50, 85 and 100 mm, and must answer within a twentieth of a degree of one another. Changing a format in one module fails both suites until the other follows.

On Set — Production, Dailies & Today

This is the part of the platform read standing up, outdoors, with something else already in your hands. Three pages carry the shooting day: `/dailies/` logs the takes, `/today/` is the day in your pocket, and `/production/` turns both into paperwork — the only record of what the day actually did, which is a different document from the one that said what it would do.

> **On the accuracy of this chapter.** It was written by reading Cinamate's source, not by working a shooting day with it. The platform is under development while this manual is being printed. Where the two disagree, the software is the authority.

One day, one record

A day means three things here — an index on the stripboard, a calendar date in the day planner, a date and unit on the logger — and until they were joined, none could find the others. The join is `SB_ShootDays_v1`, in `js/lib-shootdays.js`, holding `dayIdx`, `date` as `YYYY-MM-DD`, `unit`, the day's `sceneIds`, `wrapped` and `pinned`.

Two points of it matter on the floor. First, **a day is keyed on date *and* unit**, never the date alone: main unit and a splinter both shooting the 14th are two days' work, two call sheets and two records, and the date alone let the second silently overwrite the first. Second, **a take joins to a day on the date** and on nothing else — matched by exact string equality, across both take stores. A take carrying no date is on no report at all, deliberately, because guessing that it belongs to today is the bug this replaced; undated takes are counted separately and said out loud.

That date is the production's **local** calendar date. Wrap at 23:50 on the 7th belongs to the 7th — the day the crew worked, the day on the call sheet.

Logging a take

`/dailies/` runs in the order the day happens. Section 1 sets the shoot day and the unit (`MAIN`, `2ND`, `SPLINTER`) and shows which record that date resolves to, or *not on the stripboard yet*. Touching either control writes the record and marks it `pinned`, so a date a human stated is not recomputed by a later rebuild.

The take form is deliberately large-target: scene, slate, take, an A/B camera pair, lens, sound roll, TC in, an NG reason from a fixed list, and a note. Type a scene number and the slate and take fill themselves in from what you have already shot — same scene, same slate, next take number; a fresh scene starts at slate `A`, take 1. **NEW SETUP** bumps the setup letter the way a 2nd AC chalks it: 12A to 12B, 12Z to 12AA, the letters being bijective base-26. **+ TAKE** logs it; **● CIRCLE LAST** circles or un-circles what you just logged. On a phone those two sit in a fixed bar at the foot of the screen.

> **The ✕ on a take row deletes it at once.** No confirmation, no undo: the row leaves `SB_Dailies_v1` with its circle. Note also that the table shows only this page's own takes — rows logged in Tools → Slate live in `SB_TakeLog_v1` and count towards every report and rate below, but cannot be edited here.

The circle

A circled take is the print order, written in two alphabets. This page stores a real boolean; Tools → Slate stores the `<select>` display string `Circled ⭕` — the word *Circled* followed by the red-circle emoji — in a field called `grade`. `js/lib-shootdays.js` translates the second into the first once, so every consumer gets one answer. **`Good` is not a circle**: the grade list offers both, so an AD who chose *Good* chose it on purpose. Section 5 gives the circle rate overall and per day, and **Send picks to Editor** writes the circled takes to `SB_DailiesPicks_v1` as a pull list, replacing whatever was there.

Two logs, two clocks

This page stamps a take with the local date; `tools/tools-core.js` stamps `SB_TakeLog_v1` in UTC. When the two dates differ — west of Greenwich, most of the evening — takes logged on the phone slate file under tomorrow and will not join tonight's day. `/dailies/` prints a warning saying so; until the writers agree, log late takes here.

Scheduled and not shot

Section 4 answers the question actually asked at wrap: *of the scenes scheduled for today, which did we not get?* The stripboard says what was scheduled on a day index, the shoot-day record turns that index into a date, the take logs say what was shot on it. It needs a stripboard and a screenplay in this browser, and names whichever is missing.

A day is judged only once it is over — marked **wrapped**, or its date has passed; a scene scheduled for next Tuesday is not a gap. Results are separated by kind, because they are different facts: scheduled and never shot anywhere, with the outstanding pages in eighths; missed on the day and picked up since; shot on a day it was not scheduled for; and the joins that failed — strips matching no scene, takes against a scene number the screenplay lacks, scenes on no strip at all. **Copy the gap list** puts all of it on the clipboard for the production report.

Calling the wrap

Marking a day wrapped is the most consequential thing done on these pages. `wrapped` is the flag the schedule's pace model gates on: the board ships a planning default of 4.5 pages per day and replaces it with the median of what your *wrapped* days actually achieved, once three of them exist. A production that never wraps a day reads the shipped default for forty days and learns nothing from itself — and the pace chip says exactly that, in the words *nothing learned yet (no wrapped days)*. Wrapping also turns on the gap list above.

The control is on `/today/`, beside the day selector: ◉ **Wrap day**, becoming ↺ **Re-open day** once set. It asks first, names the day it is about to wrap, and is reversible through the same call — a day wrapped by mistake would otherwise teach the schedule a pace nobody worked. Two things established by reading: this page is the **only** place in the shipped tree that sets the flag, and `scripts/test_wrapgate.mjs` fails the build if that stops being true; and the stripboard *displays* a `WRAPPED` chip but does not toggle one, whatever its own comment says. Wrap the day on set, on the day.

> **The wrap button only offers days the stripboard knows.** `/today/` builds its day selector from the strips on `SB_ScheduleBoard_v1`. With no board in this browser there is no selector and no wrap control, and a day created only by `/dailies/` — a date the board has never heard of — falls outside the selector's range and cannot be wrapped from here.

What `/today/` shows

One day, phone-sized, found by lookup rather than by guessing at a date string. If today is a shoot day you get today; otherwise you get the next day that has not wrapped, headed `NOT TODAY` so the two cannot be confused. A wrapped day is headed `WRAPPED`.

The sheet carries the production name; the call time typed on the stripboard's call-sheet panel; day number, weekday, date and unit; the page total in eighths; the day's scenes with day/night and page count; the cast working; the sets parsed from the sluglines; the nearest hospital from the Scout Book; the safety checklist for today's scene numbers; and the call sheet's notes. Where the hospital is not on file it says so and points at the Scout Book, because it belongs on every call sheet.

The daily production report

Production Office → **Daily Report** assembles a DPR for one date from what the platform already holds; nothing on it is retyped. Pick a date — it opens on today if today is a shoot day — add notes, and build. The report heads itself with date, day number and unit, or says *not a scheduled shoot day* rather than quietly reporting the whole schedule as one day's work. It then gives the scenes scheduled on that day's strips and how many were shot, the scenes NOT shot by number, every scene covered, the take count with the circled count beside it, the crew timecard count and hot costs. 🖨 **Print** and ⬇ **.txt** are the outputs.

Two figures on it are wider than the day, and are labelled by their wording rather than by their arithmetic: hot costs are every posting to date, not the day's, and while timecards are filtered to the date, one carrying *no* date is counted on every report you build. Where the take logs hold undated takes, the report prints the count and tells you to date them.

Continuity, camera and sound

The Continuity pane keeps one row per scene and setup in `SB_Continuity_v1` — circled take, screen direction, wardrobe and props, matching notes. Beside it the Camera and Sound panes keep the classic per-roll registers, down to lens, stop, filter, timecode and mics. `/dailies/` also exports a camera report and a sound report for any one day, columned like the paper ones with circled takes marked ●. Both print the same caution: cross-check them against the camera assistant's and the sound mixer's own sheets before lab or DIT turnover.

Cards and footage

Footage is described here, never held here: a take records its camera, lens, sound roll and TC in, and the registers record the roll. The one place the platform touches media itself is Tools → **Offload Verification**. Pick a card's files and each is SHA-256 hashed in this browser, producing a manifest to keep with the media; load that manifest later, re-pick the copied files, and each comes back ok, changed, missing or extra — a clean result reported as bit-

perfect. Nothing is uploaded; the hashing happens on the device. The manifest is Cinamate's own XML, rooted at `<cinamatemanifest>`: MHL-shaped, but not an ASC MHL file, so do not hand it to a DIT expecting their tool to read it.

Sides

> **What this platform calls sides is not a redacted side.** Both builders were read for this chapter, and both emit the *complete scene*: the slugline followed by the entire body — every other character's dialogue, and all the action — for each scene selected. Nothing is withheld or blacked out. The Casting Office (`casting/lib-castdesk.js`) selects the scenes in which the character actually speaks; the Production Office (`production/lib-prod.js`) selects scenes the character *appears in* — speaking, or named in an action line — because an actor who crosses a room with no dialogue is still called that day and still needs the pages. The two builders answer deliberately different questions, and both match whole words only: a character called AL is not pulled into every scene containing CALL. The file that comes out of **⬇ .txt** is an unreleased extract of the screenplay: sent unedited to a performer's representative, it sends those pages in full. Cut what the room should not see before it leaves your hands — as the Casting Office's own header says, trim for the room and verify against the current draft before sending.

What these pages do not do

- **They do not upload footage anywhere.** `/dailies/` and `/today/` make no network calls at all; the take logs, reports and DPR are assembled in the browser from this device's own storage.
- **They notify no one.** Nothing emails a call sheet, texts a unit or tells the office a day has wrapped. Every output is a printout, a `.txt` download or the clipboard, and it moves because a person moves it.
- **None of it replaces the script supervisor's own notes.** It is a parallel record, valuable for the joins it makes across the schedule and the take logs — and its own exports say as much.
- **They are not offline apps.** The service worker caches only the public shell, by an allow-list these pages are not on, so nothing serves them without a connection. The data they read is local to the browser that logged it: a take logged on one phone is on that phone.

The Editor I — Cutting

Everything in this chapter happens in your own browser. The bin, the sequence, the trims and the playhead are held on the machine in front of you; no frame is uploaded in order to cut it, and nothing here needs a network. That is the Editor's great convenience and its one real hazard.

> **On the accuracy of this chapter.** It was written by reading the Editor's source, not by operating a finished product. Cinamate is still being built. Where this page and the running software disagree, believe the software, and treat the disagreement as a fault in the manual.

Two editors carry the name

The full Editor is `/editor/`, whose page loads the cut engine `editor/lib-cut.js` (publishing `window.CCut`), the MP4 writer `editor/lib-mp4.js` and the cutting room itself, `editor/cut-ui.js`. Three tracks, titles, undo, a persistent bin. That is what this chapter documents. The smaller editor in `editor/timeline-engine.js` is mounted inside the Studio at `/timeline/`, in the collapsible *Editor* panel: one video lane, no titles, no undo, its own storage key. Work done in one does not appear in the other.

> **Destructive.** In the Studio's Editor panel, *Sync clips* and *Pro Cut* both begin by emptying that panel's bin and edit track and rebuilding them from the Studio's clip list. There is no undo there, so trims made by hand are gone. The full Editor is unaffected — it does not share that state.

Getting media in

The bin fills from two places, and the difference matters more than anything else on the page.

Load Studio clips reads `SB_Timeline_v1`, takes every clip that has a `videoUrl`, and adds one row each, named `SC01`, `SC02` and so on with the clip's label appended. The row records the *address* of the render; the bytes are not copied.

Import files opens a picker accepting `video/*` and `audio/*`. Each chosen file is written into IndexedDB before anything else happens, then added to the bin under an id of the form `f_c…`. The bytes are copied, and they are yours.

Either way the source is then probed by a detached element for duration and, for picture, dimensions and a thumbnail. A source that refuses a pixel read — a cross-origin render, typically — yields no thumbnail; one that errors outright is marked `missing`.

Where the media actually lives

Imported files go into IndexedDB: database `cinamate_cut`, version 1, object store `media`, out-of-line keys — the key is the bin id, the value is the raw `Blob`. On the next visit each flagged row is read back and given a fresh

object URL; a row whose blob has gone is marked missing and says so, worded differently for a file than for a link.

The cut itself does *not* live there. It is written to local storage under `SB_Cut_v1`: the project object, a stripped copy of the bin (id, name, kind, duration, dimensions, origin and the `idb` flag — the object URL is deliberately blanked for rows held in IndexedDB, since an object URL dies with the tab), and a note of the last export. A production's writing and its pictures therefore sit in two different stores, and they travel differently.

What that means for backup

The vault knows about the store. `projects/lib-vault.js` carries a list called `BLOB_DBS`, and `cinamate_cut/media` is on it, labelled *cutting-room source*. A row is claimed for a production by reference: the vault reads `SB_Cut_v1`, walks its `bin` list, and takes the id of every row whose `idb` flag is set. Two consequences follow.

- An **imported file** travels. Projects → Export backup packs its bytes into the `.cinamate` archive; Import backup writes them back into `cinamate_cut/media` on the receiving machine.
- A **Studio clip** does not. It carries no `idb` flag because it is a link and there are no bytes to carry. It restores as an address, and whether that address still resolves is not a question about your backup.

Two ceilings apply. The vault's media adapter will not read any single record larger than 16 MB into memory; such a record is excluded from the archive's `blobs`, listed under `blobsMissing` instead, and the export message states how many files "had no bytes on this machine and are NOT in it". The studio cloud is far tighter — 900 KB per record, 48 MB per production — and refuses the whole push rather than truncating it. A source master is not going to the cloud.

> **An older cloud version comes back without its media.** The cloud keeps eight superseded copies of a production in a rotating ring, but the ring holds the written record only. Media parts live under separate keys and are never versioned — the restore endpoint says so in its own reply, `mediaVersioned: false`. Restore last Tuesday's version and you get last Tuesday's cut list pointing at whatever media is currently on file; bin rows may come back missing. If a cut matters, export a `.cinamate` backup. That is the only path carrying picture and cut list together.

The bin

Rows show a thumbnail, a name and a duration, with Studio rows marked · *studio*. Double-click a row to append it to the timeline, or drag it onto a track. A missing source keeps its row, marked, and refuses to be added — a cut is a list of references, and a row that quietly disappeared would leave the sequence pointing at nothing while looking shorter than it is. Nothing in the source removes a row: within one cut, the bin only grows.

The timeline model

A cut is a plain object with a name, a frame rate, a frame size and three lists.

```
video[]   {id, srcId, label, in, out, speed, trans:{type,dur}, color}
titles[]  {id, text, sub, start, dur, pos, size}
audio[]   {id, srcId, label, start, in, out, gain}
```

`in` and `out` are positions in the *source*, in seconds. A video clip does not record where it sits on the timeline, and that omission is the design.

Effective duration, and the ripple

A clip's length on the timeline is not `out − in`. It is

```
effDur(c) = max(0, (c.out − c.in) / (c.speed || 1))
```

so a four-second selection played at 2× occupies two seconds. Start times come from walking the video list and accumulating those lengths: clip *n* begins at the sum of every effective duration before it. Nothing stores a start time, so nothing can disagree with one. Trim the head of clip 1 and every clip after it moves. That is the ripple, and it is not a mode you can switch off.

The title and audio tracks work the other way: each entry carries an absolute `start` and stays where it is put. This is the trap. Retrim the picture and your titles and music do *not* follow — a title cued to a line of dialogue drifts the moment you tighten anything upstream of it, and only your eye will catch it. Overall length is the longest of the three.

Transitions

A transition belongs to the clip it plays *into*. `cut` has no duration; `crossfade` holds the outgoing clip's last frame over the incoming one at falling opacity; `fadeblack` is split down the middle, bringing the incoming clip up from black over its first half and darkening the outgoing clip's tail over the same half-length, so a dip reads across the join rather than only after it. The inspector allows 0 to 3 seconds, defaulting to 0.5 s. Titles fade over a fixed 0.3 s and cannot be changed.

Trimming, splitting, rippling

Every clip on every track has a handle at each end. A drag converts the pointer's travel to seconds by dividing by the current zoom, then multiplies by the clip's speed, because a pixel of timeline is `speed` seconds of source. Dragging the left handle of an audio block also moves its `start`, so it trims from the head instead of sliding. Each drag is clamped as it goes: `in` is forced to zero or above, `out` is clamped to the source's known duration, and a floor of 0.1 s holds between them — a clip cannot be trimmed to nothing. Where the source duration is unknown, the upper clamp is not applied.

Split (the button, or `S`) cuts the video clip under the playhead in two: the first half takes the playhead's source time as its new `out`, and a second clip begins there with the same source, label and speed and a plain cut into it. A split refuses within 0.05 s of either end of a clip, and leaves total duration unchanged — which the checks in `scripts/test_cut.mjs` assert.

Reordering is a drag of a clip onto another in the same track. On the video track that reorders the list and the ripple recomputes; on the other two it sets a new absolute start.

> **Destructive.** *Remove clip* in the inspector, and the `Delete` and `Backspace` keys, delete the selected item immediately with no confirmation. Each takes an undo snapshot first, so `Ctrl+Z` brings it back — provided you have not since spent the sixty-step history.

Frame rate, and why timecode is counted in frames

The rate is a property of the *cut*, not of the export dialogue. The picker writes `project.fps` the moment it changes, and the ruler, the timecode readout and every turnover file are counted from that one number. One function, `normFps()`, decides what qualifies as a rate: anything finite and positive is accepted, 23.976 included, and anything else becomes 24. One default, in one place, so a cut that has never had a rate written to it cannot pick up a different assumption in each exporter. The picker offers 24 and 30; a saved cut carrying a rate this build has no option for is corrected to match the picker rather than the two silently disagreeing, so check the rate when you open an old cut.

The important part is the arithmetic. Timecode comes from `tc(seconds, fps)`, in this order:

```
rate   = max(1, round(fps))         // non-drop counts the NOMINAL rate
totalF = max(0, round(sec × rate))  // ROUND ONCE, INTO FRAMES
f = totalF % rate
s = floor(totalF / rate) % 60
m = floor(totalF / (rate × 60)) % 60
h = floor(totalF / (rate × 3600))
```

Every field is derived from a single frame count, and nothing is rounded to whole seconds on the way. That is the whole point. Round to seconds first and every sub-second trim is discarded before the frames field is even computed: a half-second handle becomes zero, a twelve-frame overlap becomes nothing, and a run of small trims drifts silently against the picture it is meant to describe. The test suite pins this — `tc(0.5, 24)` must be `00:00:00:12` and `tc(1/30, 30)` must be `00:00:00:01`. The same discipline governs the interchange formats, where each lane keeps its cursor in frames and rounds once rather than accumulating rounded seconds. A fractional rate still prints against the nominal integer: at 23.976 the frames field runs 00–23 and one hour of media reads `01:00:00:00`. This is non-drop timecode, labelled as such.

> **What frames-based timecode does not do.** The model stores `in`, `out` and `start` as floating-point seconds, and nothing in the source snaps them to frame boundaries. A trim handle can land at 4.3708 s and will stay there. The frame count is computed at the moment a number is displayed or written out, so what you read is always frame-accurate to the rate — but two operations that round separately can land a frame apart. If a cut must be frame-exact, set the rate before you trim and type figures into the inspector rather than dragging them.

Audio

Audio arrives two ways. A video clip brings its own — the preview element is unmuted, so picture and production sound play together. An audio file imported into the bin lands on the A1 track as a free-positioned block with its own in and out points and a gain from 0 to 1. Which track an item goes to is decided by the bin row, not by where you drop it: a source is classified once, at import, by MIME type, and an audio row always lands on A1, a picture row always on V1, wherever the pointer released it.

In playback the A1 blocks are driven by their own hidden media elements, started when the playhead enters the block and nudged back if they drift more than a quarter of a second. Waveforms are drawn lazily — the source is fetched, decoded, reduced to 240 peak buckets and cached — and a source that cannot be decoded is marked as having no waveform and not attempted again that session. A blank strip therefore means the decode failed, not that the audio is silent.

Undo

Undo is a snapshot stack. Before each mutating action the whole project is serialised to JSON and pushed; the stack holds sixty entries and discards the oldest beyond that. Redo is cleared whenever a new snapshot is taken. A field committed without actually changing anything has its snapshot popped again, so the history does not fill with no-ops.

Two limits are worth stating plainly. Undo restores the *project* — the three tracks, the name, the rate. It does not restore the bin, so it will not bring back a source, and it clears the current selection. And it is held in memory only: it does not survive a reload, however recent the change. `Ctrl+Z` and `Ctrl+Y` (or the Command key) are wired, as are the two toolbar buttons. `Ctrl+Shift+Z` is not.

What is saved, and when

There is no Save button, and none is needed. Every action that changes the cut writes `SB_Cut_v1` immediately: adding to the bin or a track, a trim released, a reorder dropped, a split, a title added, an inspector field committed, a rename, a rate change, an undo, a redo. A write that fails — a full quota, most often — is swallowed silently, and that is the one case where the page will not tell you something went wrong.

What is *not* saved: the playhead, the zoom, the selection, the undo history and the object URLs. All are rebuilt from zero on the next load, with the playhead at the start.

What the Editor is not

It is not a finishing tool. The per-clip colour controls are four sliders — exposure, contrast, saturation, warmth — compiled into a canvas filter of `brightness`, `contrast`, `saturate` and either `sepia` or `hue-rotate`. No curves, no per-channel control, no windows, no keyframes, no scopes; a setting applies flat across the whole clip for its whole length.

It does not conform to a master. There is no relink step and no concept of a camera original. What the Editor previews and what it renders are the same files that are in the bin: Studio renders at whatever size they were generated, or the files you imported. Turning over to a finishing suite is what the interchange formats are for, and they are the next chapter's subject.

It is not a substitute for a colour session. The export path is an 8-bit H.264 encode off a canvas, tagged Rec. 709 limited range — no log pipeline, no LUT, no wide-gamut or high-dynamic-range handling anywhere in it. Grade here to make an assembly readable, then grade properly elsewhere.

Keyboard

Shortcuts are ignored while a field has focus, so a clip name containing the letter *s* can be typed without splitting anything.

KEY	ACTION
`Space`	Play or pause at 1×.
`K`	Play or pause at 1×, the same as Space.
`L`	Play; press again to shuttle 2×, then 4×, then back to 1×.
`J`	Step the playhead back one second. It does not shuttle in reverse.
`S`	Split the video clip under the playhead.
`Delete` / `Backspace`	Remove the selected clip, title or audio block.
`Ctrl`/`Cmd` + `Z`	Undo.
`Ctrl`/`Cmd` + `Y`	Redo.

The zoom slider runs from 20 to 240 pixels per second and opens at 70. The ruler is clickable along its length to place the playhead; its labels show minutes and seconds, while the transport readout carries full timecode from hours to frames. Review notes sent from the Screening Room stand on the ruler as markers.

The bin's Assistant buttons — rough cut, tighten, cut to beats — and every export and delivery format are the subject of the next chapter.

Tools & Utilities

Every production accumulates arithmetic that belongs to nobody in particular. What time does the light go. What does a fourteen-hour day cost once the meal was late. Is the copy off the card actually the same file. Who has the call sheet, and did they say so. The Tools module is where that arithmetic lives — nineteen tabs behind one rail at `/tools/`, sharing four calculation engines and one table component between them.

> **On the accuracy of this chapter.** It was compiled by reading the source files, not by operating the software. Cinamate changes week to week; if a screen contradicts a sentence here, the screen is the authority and this page is out of date.

How the module is assembled

The page is `tools/index.html`. Its rail groups the tabs five ways — *On set* (Slate & Takes, Timecards, Offload/MHL, Dailies Review), *The office* (Crew & Call Sheets, Hot Costs, Insurance, Rights / Chain of Title, Finance Waterfall), *Story & look* (Script Revisions, Story Bible, Moodboard, Lens & Coverage, Look / LUTs), *Finishing* (Captions, Credit Roll) and *Selling* (Festivals, Buyers & Investors, Press Kit). A tab builds itself the first time you open it and not before; the current tab is written into the URL fragment, so `#captions` reopens where you left off.

Underneath, the split is deliberate: *lib-* files hold arithmetic that can be tested in node with no browser at all, and *tools-* files hold the screens that call it. `tools/lib-media.js` (TMedia), `lib-sun.js` (TSun), `lib-money.js` (TMoney) and `lib-script.js` (TScript) are the four engines; the four `tools-*-ui.js` files and `tools-registers.js` are the screens.

The header of `tools/tools-core.js` states a hard load order, and it is enforced rather than trusted. The page must load `/js/safe-url.js`, then `/js/ui-chrome.js`, then `/js/ui-table.js`, and only then `tools-core.js`. `TCore` is not a second implementation of anything — it is a lazy re-export of eleven helpers from `CTable` plus `toast` from `CChrome`, each a getter that throws by name if its file is missing, so a broken load order reports the file to add instead of failing three modules later as an undefined function.

The Registers

Eleven of the nineteen tabs are built on one component — thirteen registers in all. A **Register** is defined in `js/ui-table.js` and takes a schema: a storage key, a list of fields, and optionally a summary line, a blank-row shape, an expiry field and a flags function. Each field declares a type — `text`, `money`, `number`, `date`, `select` or `textarea` — and that decides how the cell is edited. Rows persist to `localStorage` under the schema's key on every change; **+ Add** puts a new row at the top. Where a schema names an `expiryField`, that date column grows a chip: days remaining once inside thirty, and **EXPIRED** once the date is past. Insurance keys that to `expiry`, Rights to `termEnd`, Festivals to `deadline`.

REGISTER	STORAGE KEY	WHAT IT HOLDS
Crew directory	`SB_Crew_v1`	Name, role, dept, union, rate, phone, email, dietary, emergency contact
Call-sheet distribution	`SB_CallDist_v1`	Shoot day, recipient, sent via, sent date, status, notes
Insurance & certificates	`SB_Insurance_v1`	Type, carrier, policy no., limits, additional insured, dates, premium
Rights / chain of title	`SB_Rights_v1`	Work, agreement type, counterparty, territory, media, term, fee, status
Buyers & investors	`SB_Deals_v1`	Contact, company, type, territory, stage, value, last contact
Festival submissions	`SB_Festivals_v1`	Festival, category, tier, deadline, fee, result, premiere requirement
Take log	`SB_TakeLog_v1`	Shoot day, time, scene, take, roll, grade, note
Review notes	`SB_ReviewNotes_v1`	Clip, timecode, note, status
Timecards	`SB_Timecards_v1`	Date, name, role, rate, call, wrap, worked, total
Hot-cost postings	`SB_HotCost_v1`	Date, account, description, actual or PO, amount
Story bible	`SB_Bible_v1`	Name, type, one-line, detail, appearances
Investor classes	`SB_FinClasses_v1`	Class, invested, premium, corridor
Deferrals	`SB_FinDefs_v1`	Deferral, amount

> **The Crew register is a file of personal data.** `SB_Crew_v1` holds a full name, telephone number, email address, dietary requirement and emergency contact for every member of your crew. Export CSV writes all of it to `Crew.csv` in plain text, with no password and no encryption. It is also an `SB_*` key, which means the vault sweeps it into every project snapshot and into every `.cinamate` archive you hand to somebody else (`projects/lib-vault.js` excludes only the two keys that carry API credentials). Treat that CSV and that archive as you would a printed crew list: know where the copies are, and do not attach either to anything you would not send to a stranger.

> **Deleting a row asks nothing.** The ✕ at the end of a Register row removes it and rewrites the store immediately — no confirmation, no undo. On the Festivals tab the consequence reaches further: that register is bound to the same store the Festival Strategist reads, so a row deleted at `/tools/#festivals` is gone from `/festivals/` too.

Why the CSV has apostrophes in it

Export CSV writes a header row of the field labels and one row per record, naming the file after the store — `SB_Crew_v1` becomes `Crew.csv`, `SB_Rights_v1` becomes `Rights.csv`. Values are quoted and embedded quotes are doubled, in the ordinary way.

Then there is one further step, and an operator who notices it should know it is deliberate. Excel and Google Sheets both treat a cell that *begins* with `=`, `+`, `-`, `@`, a tab or a carriage return as a formula rather than as text. A location note reading `-2 hours travel`, or a hostile string typed into a field by somebody who knew what they were doing, would therefore execute on the machine of whoever opened the export. `csvSafe` prefixes a single apostrophe to any such cell. Both spreadsheets strip that apostrophe on display, so the text stays readable and stays inert; nothing is removed. The guard is exercised for real — not merely grepped for — by

`scripts/test_csv_injection.mjs`, which drives `Register.toCsv` with seven live formula payloads and asserts both that none survives and that the original text is still legible.

Sun, daylight and weather

`tools/lib-sun.js` implements the standard NOAA/Meeus solar equations, and its own header claims ±2 minutes — ample for a call sheet, not a substitute for an ephemeris. It is the platform's only solar engine: `/locations/`, `/production/`, the Producer Suite and this module all load the same file. `sunTimes(date, lat, lon)` returns six moments for a calendar day: civil dawn and civil dusk (sun at −6°), sunrise and sunset (−0.833°, the refracted horizon), and the two golden-hour boundaries — `goldenEndAM` and `goldenStartPM`, both taken at the sun 6° above the horizon. `solarNoon` gives the day's peak, `daylightHours` the span between rise and set to one decimal, and `sunAngles` the sun's *direction* at each of those moments: an azimuth in degrees clockwise from true north, an altitude above the horizon, and a sixteen-point compass name. That is the number a gaffer actually asks for.

> **Timezone: the tool computes instants, you supply the clock.** Every time above is returned as a UTC instant. `fmtLocal(ms, offsetMinutes)` renders it in whatever offset it is *given*, and if it is given none it silently uses the offset of the machine you are reading on. That is correct only if you are standing on the location. The authoritative offset is the location's own civil offset, read from the Open-Meteo forecast response (`timezone=auto` yields `utc_offset_seconds`) and then remembered on the saved plan. Until a forecast has ever been fetched for that pin, the fallback is an estimate from longitude — `round(lon / 15)` hours — which is solar mean time and knows nothing of political borders or daylight saving. It can be a full hour wrong, in the direction that puts a crew on a hillside at the wrong time. Any screen using the estimate is required to say so, and the day planner labels the column `UTC±hh:mm` with the source spelled out beneath. Read that label before you publish a call time.

There is no separately named "magic hour" in the code. Golden hour is the 6° boundary; the −6° civil dawn and dusk figures are what bracket the shootable twilight either side of it.

Weather is fetched by your browser from the keyless public Open-Meteo API — daily weather code, maximum and minimum temperature, maximum precipitation probability and maximum wind. `wmoLabel` turns the WMO code into English and reports `Code n` honestly for anything it does not know. `shootRisk` scores a day out of 100: precipitation probability weighted 0.6, a maximum wind above 30 in the forecast's own unit adding up to 25 more, plus 30 for a thunderstorm code or 15 for the rain and snow codes between 61 and 86. It is a triage number for reordering exteriors, not a forecast in itself, and a failed fetch is reported as a failure — a blocked request cannot masquerade as "beyond forecast".

Money: timecards, hot costs, waterfall

`tools/lib-money.js` carries three engines. The timecard is the one an operator meets first. Give it an hourly rate, a call and a wrap, and it splits the day under the published twelve-hour conventions held in `TC_DEFAULTS`: 1.5× beyond eight *worked* hours, 2× beyond twelve *elapsed* hours, 3× golden time beyond fifteen elapsed. Note which clock each threshold reads — overtime is measured on worked hours, double and golden time on elapsed, which is the industry convention and the reason meals move one figure and not the other. Up to two meals of thirty minutes each come off the worked total.

The first meal is due within six hours of call; a late or missing meal accrues a penalty per violated half-hour on an escalating scale of 25, 30 then 50, capped at twelve half-hours. Turnaround is ten hours, and a short rest since yesterday's wrap bills the invaded hours at the crew member's rate. A sixth consecutive day multiplies every hour line by 1.5 and a seventh by 2.0. Fringes default to 28% of the subtotal. The engine's docstring is unambiguous that these are published conventions parameterised for your own agreement, and that this is neither payroll nor legal advice.

> **Only one rule is editable on the screen.** `TMoney.timecard` accepts an override for every threshold above, but the Timecards tab wires just one of them: *Fringe %*. The tab's own footnote invites you to "edit the thresholds in the fields above", and the shipped fields do not include them — call, wrap, meals, first-meal hour, day-of-week, yesterday's wrap and fringe are the whole set. A production on a different sideletter should verify the split by hand rather than assume the defaults were changed.

Hot Costs groups postings by top-sheet account code, separates spend already invoiced from purchase orders merely committed, and reports total against budget with *HOT* above 85% used and *OVER* past 100; the budget column seeds from the saved top sheet. **Finance Waterfall** layers instruments on a lifetime revenue figure: deferrals paid first, then each investor class recouping its investment plus its premium, then the remaining pool split by corridor percentages with the balance to the producer and talent pool. A step that cannot be paid in full is shown short rather than rounded away.

Script utilities

`tools/lib-script.js` holds two unrelated things that happen to both be text. The first is a longest-common-subsequence line differ behind **Script Revisions**, which snapshots the Studio's current script as a locked draft and compares any two, generation by generation, in the production colour order: White, Blue, Pink, Yellow, Green, Goldenrod, Buff, Salmon, Cherry, 2nd Blue, 2nd Pink.

The second is captions. `parseCaptions` reads SRT and WebVTT through the same path — it accepts both the comma and the full-stop decimal separator, tolerates a byte-order mark, skips `WEBVTT`, `NOTE` and `STYLE` blocks, and sorts cues by start time. `toSrt` and `toVtt` write either format back out, so the conversion between them is simply parse-then-write. `captionQc` checks each cue against the ordinary broadcast conventions and reports what it finds: reading speed above roughly 20 characters per second, a line longer than about 42 characters, three or more lines in one cue, a cue that overlaps its predecessor, and an end before its start. In the Captions tab, cues live in `SB_Captions_v1`, a line break inside a cue is typed as a space, a vertical bar and a space, and the exports land as `captions.srt` and `captions.vtt`.

Camera media: lenses, LUTs and offload verification

`tools/lib-media.js` is the module's camera maths, and it owns the platform's single sensor table — nine formats, each with its aperture in millimetres, read by the Set Designer's 2D plan and its 3D viewport as well as by the Lens tab, so one lens gives one answer everywhere.

KEY	FORMAT	KEY	FORMAT
`super35`	Super 35, 24.9 × 18.7	`alexa-65`	ARRI 65, 54.1 × 25.6
`super35-17x9`	Super 35 17:9, 24.6 × 13.1	`red-vv`	RED V-RAPTOR VV, 40.96 × 21.6
`fullframe`	Full frame, 36 × 24	`mft`	Micro 4/3, 17.3 × 13
`alexa-lf`	ARRI LF, 36.7 × 25.5	`s16`	Super 16, 12.52 × 7.41
`iphone-main` — phone main camera, ~9.8 × 7.3		Unknown or missing key resolves to `super35`	

From a format and a focal length the Lens tab gives horizontal and vertical field of view, the frame's width and height at your subject distance, and the full-frame equivalent focal length, with a top-down frustum drawn for blocking conversations. Given an aperture, it adds depth of field. The circle of confusion is stated rather than buried: frame diagonal divided by 1500, which reproduces the familiar 0.029 mm on full frame; a house working to a different standard passes its own value instead of editing the file. Distances are metres, and the far limit at or past the hyperfocal is returned as infinity rather than as a large number pretending to be a measurement.

Look / LUTs parses the public IRIDAS/Adobe `.cube` text format and previews it over a still with an intensity blend. It is 3D only — a 1D LUT is refused with a message saying so, and a truncated file is refused naming how many rows were expected against how many arrived.

Offload / MHL is the one to know on a card day. Pick the files off a card and each is SHA-256 hashed in your browser — nothing is uploaded anywhere — into an XML sidecar manifest recording path, size and digest per file. Later, load that manifest and re-pick the copied files: the verifier reports each file as ok, changed, missing or extra, and declares the copy clean only when nothing has changed and nothing is missing. That is the difference between believing a copy and knowing it.

> **What this module does not contain.** There is no media *storage* calculator here. Nothing in `/tools/` holds a codec list, a data rate in megabits, a card capacity or a drive-sizing estimate, and no such table was found anywhere in the shipped tree. Any figure of that kind must come from your camera manufacturer's own tables. The manual will not invent one, and neither should a spreadsheet built from memory the night before a shoot.

The tabs that are neither a register nor a calculator

Slate & Takes is a tappable slate that flashes, sounds a short 1 kHz tone for sync, writes the take into the log and advances the take number. Every logged take carries the shoot day as a field, because a take with no date appears on no daily production report — the summary line counts the undated ones and says so. **Dailies Review** plays a clip locally, steps a frame at a time (at a fixed 1/24 s), lets you draw on the picture and logs notes against the running timecode. **Moodboard** holds dragged and resized reference images with notes, downscaled on import to keep the store manageable, and exports the arrangement as a PNG. **Credit Roll** builds its text from the project title, the script's characters and the Crew directory, previews the scroll at one of three speeds and records it out as a WebM. The **Press Kit** tab assembles title, logline, runtime, synopsis, stills and credits into a single self-contained HTML file you can send on. All of them run entirely in the browser; nothing is uploaded.

What is verified, and how

Four suites cover this module, and they matter because they say which numbers have been checked against something outside the code. `scripts/test_tools.mjs` drives all four libraries — sun times against the NOAA calculator, the diff, the caption round-trip, the timecard split, the hot-cost roll-up, the waterfall, the LUT and the manifest. `scripts/test_media.mjs` checks the sensor table, fields of view against published lens tables, and the depth-of-field algebra — including the definitional test that focusing at the hyperfocal puts the near limit at half of it and the far limit at infinity. `scripts/test_sun.mjs` exists solely because the timezone defect was found three times and survived each time: its fixtures are all remote, and the offset assertions are re-run inside child processes pinned to three different machine timezones, because a suite that only asks about its own clock cannot see that class of bug. `scripts/test_csv_injection.mjs` covers the export guard described above.